

Schrijfgids Requirements

Versie 1.2



CleanText Academy

CleanText Academy heeft voor afstudeerders de workshop *Requirements specificeren* ontwikkeld. Deze workshop is direct gekoppeld aan *Schrijfgids Requirements*. Zowel het boek als de workshop zijn gratis beschikbaar op CleanText Academy:

> <https://cleantext-academy.blogspot.com>



Aanbevelingen

“Om snel en flexibel waardevolle software voor de business te leveren, worden veel IT-systemen op een Agile manier ontwikkeld. Hierbij zijn requirements meestal in user stories beschreven en omdat deze vrijwel altijd functioneel van aard zijn, blijven de niet-functionele requirements (o.a. performance en security) onderbelicht. De auteurs van dit boek reiken de studenten handvatten aan voor het kwantificeerbaar opstellen van deze zo belangrijke niet-functionele requirements.”

- > *Leo van der Aalst, CTO bij DevOn*
Gecommitteerde bij afstudeerprojecten, spreker, trainer, auteur, ex-lector
softwarekwaliteit aan Fontys Hogeschool ICT.

“Goede requirements zijn belangrijk als je snel en effectief de juiste ICT-oplossing wilt realiseren. In de praktijk zie ik veel professionals moeite hebben om hun wensen en eisen helder, volledig en eenduidig te formuleren. Neem het ze niet kwalijk, het is ook moeilijk. De auteurs bieden de ICT-student een praktische gids om zich het requirement schrijven eigen te maken. Het is een basisgids, gemakkelijk toegankelijk en voorzien van praktijkvoorbeelden en opdrachten. Een goede laagdrempelige opstap. Dat ‘Schrijfgids requirements’ onder Creative Commons beschikbaar wordt gesteld maakt het nog toegankelijker.”

- > *Derk-Jan de Grood, Agile Coach & Test Manager bij Squerist*
Spreker, trainer, auteur

"Het boek ‘Schrijfgids requirements’ bevat handvatten voor het opstellen van requirements. In het boek is een goede balans gevonden tussen theorie en praktische toepasbaarheid. Wat ik waardeer in het boek zijn de vele voorbeelden en oefeningen waardoor het boek didactisch goed in elkaar zit. Hierdoor is het bruikbaar in het onderwijs."

- > *Albert Hofstede, Onderwijsmanager Informatica bij Hogeschool Rotterdam*

“Cursisten worden niet altijd - “to put it mildly” - enthousiast van onderwijsconcepten en de daarbij behorende theoriemethodes. Hossein, vele anderen en ik delen al vele jaren het enthousiasme voor het voorbeeldig consistente onderwijsconcept dat de wereldwijde duikorganisatie PADI heeft ontwikkeld. “Leren voor beheersing” door zorgvuldig afgestemd theoriemateriaal, toetsen en frequente daaraan gekoppelde praktijkoefeningen. Veel van deze componenten zie ik ook terug in het boek ‘Schrijfgids requirements’.”

- > *Erik Huigen, Discovery Divers*
PADI Elite Instructor, Master Instructor, Specialty Instructor en Free Dive Instructor

“De praktijk laat zien dat onduidelijke requirements vaak leiden tot misverstanden en slecht werkende software. Dit boek geeft prettig geschreven handreikingen hoe dit te voorkomen. En deze handreikingen zijn niet alleen nuttig voor studenten maar zeker ook van toepassing op onze dagelijkse werkzaamheden als ICT'ers. Een aanrader dus om te lezen maar vooral ook om daadwerkelijk te gebruiken.”

- > *Okke van 't Verlaat, CTO bij NGTI*
Gecommitteerde bij afstudeerprojecten, afstudeerbegeleider

Dankwoord

Als auteur ben ik dank verschuldigd aan allen die een bijdrage hebben geleverd aan de totstandkoming van dit boek. In het bijzonder wil ik de volgende personen danken voor hun kritische blik en constructieve bijdragen:

- > Egbert Bouman, Test en agile specialiste bij Valori
- > Bob Eleveld, Hogeschooldocent bij Hogeschool Rotterdam
- > Xenia Hasker, Hogeschooldocent bij Hogeschool Rotterdam
- > Erik Kuiper, Docent informatica bij Avans Hogeschool
- > Bert Luppens, IBM Executive Project Manager / Complex Program Manager
- > Nassim Mehrshid, Solution designer bij ABN AMRO
- > Erik Rübsamen, Docent bij Zuyderzee Lyceum Emmeloord
- > Robert de Ruiter, Gastdocent bij Hogeschool Rotterdam, oud-hoofdredacteur .NET Magazine, Java Magazine, Release en Optimize bij Array Publications bv

Van meet af aan werd ik ondersteund en gestimuleerd door mijn werkgever Hogeschool Rotterdam. De ondersteuning van de hogeschool was onmisbaar.

Schrijfgids requirements

Hossein Chamani

In samenwerking met Anne van Brussel en Jos van Rooyen als sparringpartners.

Redactie: Robert de Ruiter

CleanText Academy

Inhoud

Voorwoord	7
Introductie.....	8
1 Requirements in een notendop	12
2 Functionele en niet-functionele software requirements	18
3 ISO 25010 Kwaliteit van softwareproduct.....	22
4 Handreikingen voor requirements schrijven	36
5 Onethisch en manipulatief gedrag herkennen	50
6 Vrije opdrachten gericht op HBO-i beroepstaak software analyseren	54
Bijlage A: CleanText kwaliteitskenmerken.....	61
Bijlage B: SRS-template voor een (afstudeer)project	63
Bijlage C: Voorbeeld use case beschrijving	66
Bibliografie	67
Over de auteur.....	69
Download gratis boeken en workshops.....	70

Voorwoord

Ik ben altijd al een fan geweest van de strip Dirk-Jan. Laatst las ik weer een stripje van hem en ik moest lachen én aan het belang van requirements denken. Op het eerste plaatje zie je Dirk-Jan die via Tinder met een dame chat. Zij vraagt aan Dirk-Jan iets over zichzelf te vertellen en hij zegt dat hij van pizza houdt en make-up draagt. Zij ‘ontmatcht’ Dirk-Jan meteen. Op het laatste plaatje zie je Dirk-Jan dozen met oogschaduw, lippenstift en highlighter een drogisterij in dragen. Communicatie...

Het belang van goede requirements kan niet voldoende benadrukt worden. Bij complexe systemen of systemen waar meerdere gebruikers mee werken is degene die weet wat het systeem moet doen en degene die het systeem maakt doorgaans niet dezelfde persoon. Daarnaast hebben soms meerdere personen of organisaties informatie over het systeem nodig. En dan moet er dus informatie uitgewisseld worden tussen verschillende mensen met verschillende achtergronden. Of zoals op pagina 36 van dit boek staat: ‘Zonder informatie kun je geen software ontwikkelen voor en met anderen.’

Maar hoe ziet die informatie eruit? Wat zijn dan ‘goede’ requirements? Deze schrijfgids requirements geeft studenten én professionals in het veld de nodige handvatten hiervoor. De vele voorbeelden en de opgaven maken het boek in hoge mate praktisch en toegankelijk.

Het is bijzonder goed te constateren dat er ook in opleidingen aandacht wordt besteed aan het opstellen van requirements. ICT-opleidingen besteden veel aandacht aan techniek en dat is belangrijk en goed. Maar als je fantastische, gestructureerde, efficiënte en onderhoudbare software maakt maar de software doet niet wat het volgens de opdrachtgever en gebruikers móét doen, heb je nog steeds niets bereikt. Kennis van requirements helpt hen om het juiste product op de juiste manier te ontwikkelen voor de beoogde opdrachtgevers, gebruikers en beheerders.

En Dirk-Jan? Ik heb al veel stripjes van hem gelezen en hij is niet zo heel gelukkig met vrouwen, om het vriendelijk te zeggen. Hoe dat komt? Communicatie. Het slechte nieuws voor Dirk-Jan is dat deze schrijfgids requirements hem niet zal helpen, relaties zijn toch wat anders dan software ontwikkelen. En eigenlijk weet ik ook niet hoe we Dirk-Jan kunnen helpen. Maar voor studenten en professionals die in ICT-projecten werken heb ik wel goed nieuws; deze schrijfgids helpt ze om succesvol(ler) te worden.

*Jan Jaap Cannegieter,
Principal Consultant Squerist bv
Spreker, trainer, mede-auteur ‘Succes met requirements’ en ‘Grip op requirements’.*

Introductie

Is software het eindproduct van je afstudeerproject?

Ben je als ICT-student in staat om de relevante requirements consistent, eenduidig en duidelijk te formuleren voor de belanghebbenden?

Het onderzoeken van requirements is één van de eerste en belangrijkste taken in je afstudeerproject. De door jou geschreven requirements bieden je docenten de benodigde informatie om te bepalen of je opdracht geschikt is voor een afstudeerproject. Heeft de opdracht het juiste niveau? Is de opdracht realiseerbaar binnen de afstudeerperiode? Jouw requirements geven jou, de opdrachtgever en de begeleiders van het afstudeerproject inzicht, om samen de opdracht aan te passen als het nodig is.

Requirements zijn ook het uitgangspunt waarmee je tijdens de beoordeling van je afstudeerproject aantoont dat je de juiste software op de juiste manier hebt ontwikkeld voor de opdrachtgever, gebruikers, beheerders en andere belanghebbenden. Je laat zien dat je prestaties voldoen aan de eisen van de opleiding.

Dit boek is speciaal geschreven voor ICT-studenten en ICT'ers in opleiding. Het boek dient als gids voor het schrijven van requirements. De belangrijkste bouwstenen zijn praktische concepten die worden versterkt door voorbeelden, oefeningen en opdrachten. Deze eigenschappen maken het boek geschikt voor zelfstudie, trainingen en gebruik in projecten tijdens de hele opleiding. Daarnaast verwijst het boek, bij elk belangrijk onderwerp, naar additionele kennisbronnen. De studenten kunnen zich verder verdiepen in de materie door deze bronnen te gebruiken.

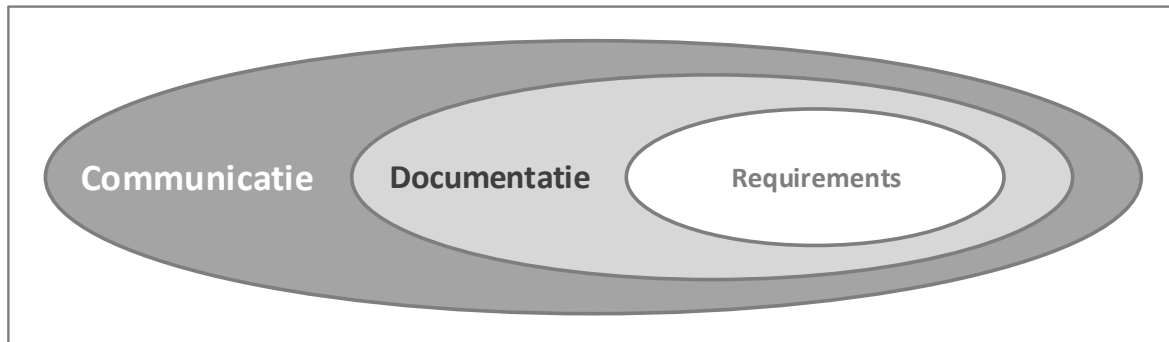
Schrijfgids requirements is ook bedoeld als ondersteunend materiaal voor:

- Communicatiedocenten die het schrijven van requirements willen integreren in het taal- en projectonderwijs, zodat studenten zich beter bewust worden van de waarde van de taal bij het uitwisselen van informatie in het ontwikkelproces van software.
- ICT-docenten die het schrijven van requirements willen integreren in het ICT- en projectonderwijs, zodat studenten leren requirements als uitgangspunten te gebruiken voor taken zoals het ontwerpen, ontwikkelen, testen en in gebruik nemen van software.
- Opdrachtgevers en afstudeerbegeleiders bij bedrijven en instellingen die een ICT-student willen ondersteunen bij het opstellen van requirements, zodat alle betrokkenen een expliciet en onderling afgestemd uitgangspunt hebben voor het begeleiden en waarderen van de student in het afstudeerproject.

Waarom leren requirements te schrijven?

In de praktijk gebruiken ICT'ers software requirements als informatiebasis voor taken zoals het ontwerpen, ontwikkelen, testen, in gebruik nemen, beheren, onderhouden of selecteren van softwareproducten. Requirements zijn ook een vorm van documentatie voor de formele of informele afspraken over de gewenste functionaliteit en kwaliteit van software.

Ze vormen één van de belangrijkste communicatiemiddelen als de opdrachtgevers, gebruikers en ontwikkelteams verspreid zijn over verschillende organisaties of landen. Daarom is het belangrijk om als ICT'er leren requirements te schrijven. Figuur 1 toont de positie van requirements als kennisbron.



Figuur 1 Positie van requirements als kennisbron

Additioneel studiemateriaal en naslagwerk

Dit boek behandelt alleen concepten die je nodig hebt om requirements te leren schrijven. We raden je aan om additionele bronnen te gebruiken als je je wilt verdiepen in andere aspecten van het vakgebied requirements engineering.

Als voorbeeld willen we twee boeken voorstellen. Beide boeken bieden een solide kennisbasis die past bij een hbo-studie. Ze zijn bruikbaar als studiemateriaal en als naslagwerk voor projecten tijdens en na de studie. Kies het boek dat het beste bij jouw doelstelling past.

1. De Swart, N. (2017). *Handboek Requirements: Leidraad voor analisten in agile, traditionele en hybride omgevingen*. Tweede druk. Delft: Uitgeverij Eburon.
 - 'Handboek requirements' bevat helder geschreven hoofdstukken. Ze zijn compact en bieden tegelijkertijd de benodigde diepgang. Het boek is geschikt voor zowel een inleidende als voor een verdiepende zelfstudie of training. Daarnaast schrijft de auteur regelmatig inspirerende artikelen over de actuele ontwikkelingen in het vakgebied. Deze zijn te vinden op de volgende website:
> <https://www.reaco.nl>
2. Laplante, P.A. (2018). *Requirements engineering for software and systems*. Third Edition. Boca Raton: CRC Press.
 - Dit boek is speciaal geschreven voor studenten. De hoofdstukken hebben een conceptuele benadering met verwijzingen naar degelijke bronnen en standaarden voor requirements. Elk hoofdstuk wordt afgesloten met een reeks oefeningen. Naast de genummerde hoofdstukken bevat het boek 94 pagina's aan bijlagen met uitgewerkte requirements en diagrammen voor concrete systemen.

Studiemateriaal voor IREB- certificering

IREB, de International Requirements Engineering Board, een non-profit organisatie, is de aanbieder van het 'Certified Professional for Requirements Engineering (CPRE)' certificeringsprogramma. Een certificaat verhoogt je status bij sommige werkgevers en klanten. Daarom kan het nuttig zijn om het te behalen. Voor meer informatie, bezoek de volgende website:

> <https://www.ireb.org>

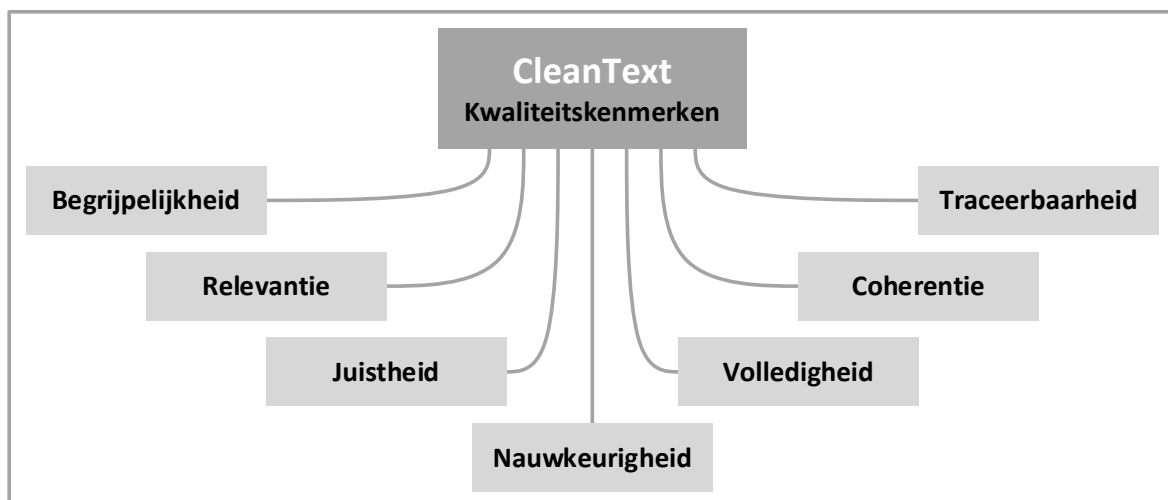
Je kunt één van de volgende twee boeken kiezen als je je wilt voorbereiden op het IREB Foundation Level Exam:

1. Cannegieter, J., De Swart, N., Zandhuis, J. (2014). *Grip op Requirements: IREB foundation examenstof uitgelegd en praktisch gemaakt*. Delft: Uitgeverij Eburon.
Op de website van dit boek vind je nog meer hulpmiddelen:
> <https://www.gripoprequirements.nl>
2. Pohl, K., Rupp, C. (2016). *Requirements Engineering Fundamentals: A Study Guide for the Certified Professional for Requirements Engineering Exam - Foundation Level - IREB Compliant*. San Rafael, CA: Rocky Nook Inc.

CleanText kwaliteitskenmerken

Fouten, onduidelijkheden of andere problemen in technische teksten kunnen leiden tot foute handelingen in het ontwikkel- of gebruiksproces van software. Dit kan fatale gevolgen hebben voor gebruikers, opdrachtgevers, leveranciers en andere betrokkenen. Daarom moet elke technische tekst aan specifieke kwaliteitskenmerken voldoen. Deze kenmerken bepaal je vanuit het perspectief van de beoogde lezers en andere belanghebbenden.

CleanText definieert zeven kwaliteitskenmerken als uitgangspunt voor het creëren van technische teksten zoals weergegeven in figuur 2. In bijlage A, CleanText kwaliteitskenmerken vind je meer informatie over deze kwaliteitskenmerken.



Figuur 2 CleanText kwaliteitskenmerken

Hij, zij en andere vormen van gender in de tekst

Omwille van de leesbaarheid is steeds 'hij' gebruikt in de tekst. Waar 'hij' staat, kunnen ook 'zij' of andere vormen van gender worden gelezen.

Opdrachten gericht op HBO-i beroepstaak software analyseren

Hoofdstuk 6 van dit boek bevat drie vrije opdrachten. Deze zijn gekoppeld aan de HBO-i beroepstaak software analyseren op beheersingsniveau 1, 2 en 3. Tabel 1 toont de beschrijvingen en de beheersingsniveaus van deze beroepstaak. De inhoud van de tabel hebben we overgenomen van de 'HBO-i-domeinbeschrijving 2018'. De volledige uitgave vind je op:

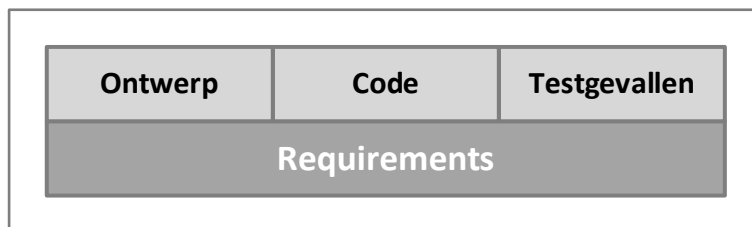
> <https://www.hbo-i.nl>

Tabel 1 HBO-i beroepstaak software analyseren beheersingsniveau 1, 2 en 3

Beheersingsniveau 1	
•	Verzamelen en valideren van functionele eisen voor een softwaresysteem met één belanghebbende volgens een standaardmethode.
•	Definiëren van acceptatiecriteria voor bovengenoemde functionele eisen.
Beheersingsniveau 2	
•	Uitvoeren van een requirementsanalyse voor een softwaresysteem met verschillende belanghebbenden, rekening houdend met de kwaliteitseigenschappen waaronder security.
•	Uitvoeren van een analyse om functionaliteit, veiligheid, ontwerp, interfaces e.d. van een bestaand systeem of bestaande component te formuleren en te valideren.
•	Opstellen van een acceptatietest aan de hand van kwaliteitseigenschappen.
Beheersingsniveau 3	
•	Uitvoeren van een requirementsanalyse voor een softwaresysteem met verschillende belanghebbenden in een context van bestaande systemen.
•	Definiëren van acceptatiecriteria aan de hand van kwaliteitseigenschappen en een uitgevoerde risicoanalyse met onder andere aandacht voor security aspecten.

1 Requirements in een notendop

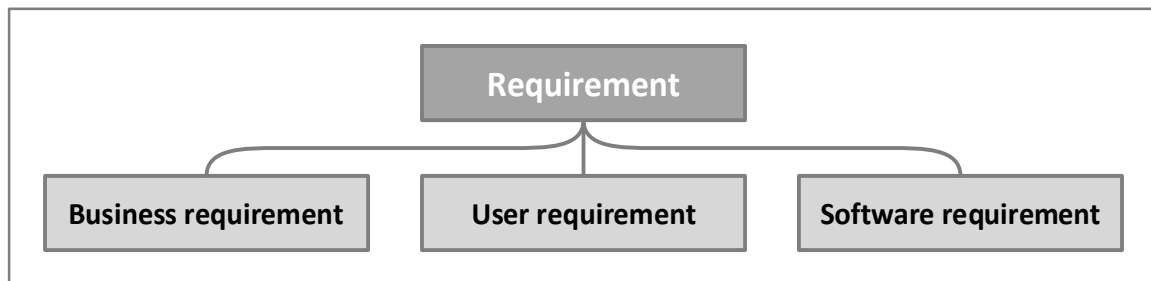
In het ontwikkelproces van software, creëren ICT'ers vier belangrijke artefacten die direct invloed hebben op de kwaliteit van het softwareproduct: requirements, ontwerp, code en testgevallen. Een requirement beschrijft een behoefte van een belanghebbende, een eis aan een systeem of een restrictie. ICT'ers gebruiken requirements als uitgangspunt voor hun ontwerp, code en test. Figuur 1.1 toont de positie van requirements als uitgangspunt.



Figuur 1.1 *De positie van requirements*

Indeling van requirements in dit boek

In de praktijk kom je diverse soorten requirements tegen. In dit boek zullen we ons alleen richten op het schrijven van business requirements, user requirements en software requirements. De kennis, vaardigheden en inzichten die je hierbij ontwikkelt, kun je even goed toepassen bij het schrijven van andere soorten requirements. Figuur 1.2 toont de indeling van requirements.



Figuur 1.2 *Indeling requirements*

Business requirement

Een business requirement beschrijft wat een belanghebbende wil bereiken met een softwareproduct of een systeem vanuit het perspectief van de business.

Voorbeeld

- De eigenaar van de koffiehandel wil in Nederland en België koffie kunnen verkopen zonder fysieke winkels, zodat hij geen tijd en geld hoeft te investeren in deze winkels.

User requirement

Een user requirement beschrijft een taak, actie of handeling die een gebruiker geheel of gedeeltelijk met een softwareproduct of een systeem wil uitvoeren.

Voorbeeld

- De eigenaar van de koffiehandel wil de klanten informeren over aanbiedingen.

- De klant van de koffiehandel wil producten bestellen.

Software requirement

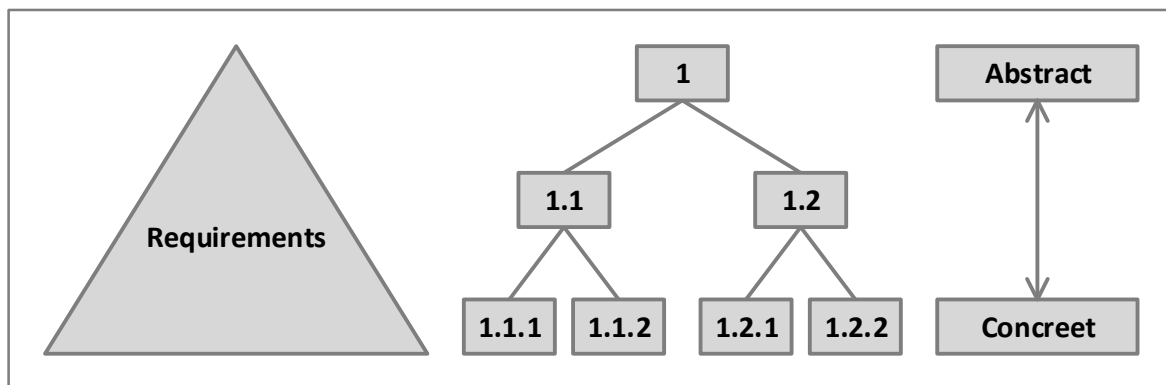
Een software requirement is een eis aan een softwareproduct of aan een systeem.

Voorbeeld

- De webshop van de koffiehandel moet het online verkoopproces ondersteunen.
- De webshop van de koffiehandel moet de klant ondersteunen om producten te bestellen.

Requirements splitsen

In de voorgaande voorbeelden zijn de requirements nog globaal beschreven. Dit hoeft geen probleem te zijn. Afhankelijk van het doel, de doelgroep en de context van het ontwikkelproces kun je globale requirements geleidelijk concretiseren totdat deze specifiek genoeg zijn. Figuur 1.3 toont hoe je een globale user requirement concreter kunt maken door deze te splitsen in specifiekere requirements.



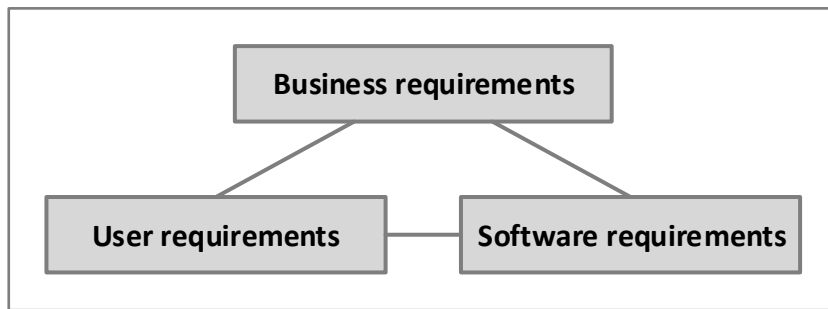
Figuur 1.3 Een globale requirement concreter maken door deze te splitsen

Voorbeeld

- 1 De treinreiziger wil een treinkaart kopen.
 - 1.1 De treinreiziger wil bij een automaat een treinkaart kopen.
 - 1.1.1 De treinreiziger wil contant betalen.
 - 1.1.2 De treinreiziger wil met een betaalspas betalen.
 - 1.2 De treinreiziger wil online een treinkaart kopen.
 - 1.2.1 De treinreiziger wil met iDEAL betalen.
 - 1.2.2 De treinreiziger wil met PayPal betalen.

Relatie tussen requirements

Softwareproducten en systemen hebben alleen bestaansrecht als de leveranciers erin blijven investeren en de klanten het blijven gebruiken. Daarom moeten software requirements afgeleid zijn van business en user requirements. Als ICT'er kun je deze drie soorten requirements gebruiken om aan te tonen dat je het juiste product op de juiste manier ontwikkelt voor de beoogde belanghebbenden zoals de opdrachtgevers, gebruikers en beheerders. Figuur 1.4 toont de relatie tussen requirements.



Figuur 1.4 *Relatie tussen requirements*

Voorbeeld

- **Business requirement**
De tandarts wil, dat het aantal cliënten dat een afspraak vergeet, van gemiddeld vijf keer per week naar nul daalt, zodat zij haar beschikbare behandeltime volledig aan haar cliënten kan besteden.
- **User requirement**
De tandartsassistente wil 48 uur voorafgaand aan elke geplande afspraak een herinneringsbericht versturen naar de beoogde cliënt met daarin de datum en de tijd van de afspraak.
- **Software requirement**
Het afsprakensysteem van de tandarts moet 48 uur voorafgaand aan elke geplande afspraak een herinneringsbericht versturen naar de beoogde cliënt met daarin de datum en de tijd van de afspraak.

User stories

In agile projecten werk je als ICT'er meestal met user stories. Een user story is ook een vorm van een user requirement. Deze bestaat uit drie delen: actor, actie, uitkomst. Hieronder zie je twee voorbeelden. In het eerste voorbeeld is *tandartsassistente* de actor. *Een herinneringsbericht versturen* is de actie. De uitkomst is: *de cliënt komt volgens de afspraak naar de tandarts*.

Voorbeelden

- Als tandartsassistente wil ik 48 uur voorafgaand aan elke geplande afspraak een herinneringsbericht versturen naar de beoogde cliënt met daarin de datum en de tijd van de afspraak, zodat de cliënt volgens de afspraak naar de tandarts komt.
- Als klant van de koffiehandel wil ik online producten bestellen, zodat deze bezorgd worden op het door mij gewenste adres.

Restricties (constraints)

In het ontwikkelproces van software kun je te maken hebben met restricties waaraan je gebonden bent. Denk bijvoorbeeld aan een afstudeerproject. Je opdrachtgever eist dat je de beoogde software in C# ontwikkelt, omdat alle andere ontwikkelaars in het team alleen met C# werken. Als je in een andere taal gaat werken, zullen ze jouw software niet overnemen.

Voorbeelden

- De ontwikkelaars moeten het simulatieprogramma in C++ schrijven.
- Het 'fleet vehicle tracking system' moet gebruik maken van het gps.

Requirements voorzien van acceptatiecriteria

Acceptatiecriteria zijn condities waarmee acceptanten kunnen controleren of het ontwikkelteam het juiste softwareproduct op de juiste manier oplevert. Een acceptant kan bijvoorbeeld een rol hebben als product-owner, gebruiker, tester, beheerder, maintenance engineer en security expert. Acceptanten kunnen op basis van deze criteria het product toetsen, testen of meten. Het toetsen, testen of meten van het product door acceptanten noemen we acceptatietest.

In tabel 1.1 hebben we acceptatiecriteria opgesteld voor een user story. Je kunt ook acceptatiecriteria opstellen voor business requirements, user requirements en software requirements. De principes blijven dezelfde. US-ID staat voor User Story Identifier. AC-ID staat voor Acceptatiecriterium Identifier. Deze ID's maken traceerbaarheid mogelijk.

Gebruik je een tool voor user stories en acceptatiecriteria? Dan bestaat de kans dat de tool een andere layout hanteert dan in het voorbeeld. Gebruik een layout die past bij de tool.

Tabel 1.1 *User story voorzien van vier acceptatiecriteria*

Topic	US-ID	User story	AC-ID	Acceptatiecriterium	Prioriteit
Bestelmodule	US1	Als eigenaar van de webshop wil ik dat de klant een besteld product direct kan annuleren wanneer hij zich bedenkt, zodat ik het product niet hoeft te versturen als het nog niet verstuurd is.	US1-AC1	De webshop heeft een optie 'bestelling annuleren'.	Vereist
			US1-AC2	De webshop stuurt een bevestigingsmail naar de klant zodra de klant de bestelling heeft geannuleerd.	Vereist
			US1-AC3	De webshop stopt het verzendproces als de bestelling nog niet verzonden is.	Vereist
			US1-AC4	De webshop maakt een werkopdracht voor de eigenaar om het betaalde bedrag terug te storten naar de bankrekening van de klant als de bestelling nog niet verzonden is.	Vereist

De acceptatiecriteria kun je ook direct koppelen aan de kwaliteitskenmerken van software. Je gebruikt bijvoorbeeld ISO 25010 als kapstok om acceptatiecriteria schrijven. Meer informatie over ISO 25010 vind je in: hoofdstuk 3 ISO 25010 Kwaliteit van softwareproduct.

Oefeningen

Oefening 1

Je hebt de opdracht om een 'GPS fleet vehicle tracking system' te selecteren voor een regionaal taxibedrijf. Om een geschikt systeem te kunnen selecteren heb je requirements nodig.

- 1 Formuleer twee business requirements vanuit het perspectief van de eigenaar van het taxibedrijf.
- 2 Formuleer twee user requirements vanuit het perspectief van een planner bij de klantenservice van het taxibedrijf.
- 3 Formuleer twee user requirements vanuit het perspectief van een passagier.
- 4 Formuleer twee software requirements.

Oefening 2

Je wilt een game ontwikkelen met een groep studenten. Formuleer een restrictie (constraint) als het gaat om de te gebruiken programmeertaal.

Uitwerkingen van oefeningen

Voor deze oefeningen kun je verschillende uitwerkingen opstellen. De onderstaande uitwerkingen zijn bedoeld als voorbeeld en vergelijkingsmateriaal.

Uitwerking oefening 1

Business requirements

- 1.1 De eigenaar van het taxibedrijf wil dat het aantal ritten per dag per taxi met 30% toeneemt.
- 1.2 De eigenaar van het taxibedrijf wil dat het aantal leeggereden kilometers per taxi met 40% afneemt.

User requirements

- 2.1 De planner wil de taxi met de kortste reistijd naar de klant sturen.
- 2.2 De planner wil de actuele locatie van alle taxi's kunnen zien.

User requirements

- 3.1 De passagier wil dat de taxi op de afgesproken tijd aankomt bij de afgesproken ophaalplaats.
- 3.2 De passagier wil de actuele locatie van de toegewezen taxi, voorafgaand aan het ophaalmoment kunnen volgen.

Software requirements

- 4.1 Het GPS fleet vehicle tracking system moet de actuele locatie van alle taxi's in een overzichtskaart weergeven.
- 4.2 Het GPS fleet vehicle tracking system moet voor elke taxioproep een beschikbare taxi vinden met de kortste reistijd.

Uitwerking oefening 2

De ontwikkelaars moeten de game in C++ schrijven.

2 Functionele en niet-functionele software requirements

In hoofdstuk 1 heb je kennisgemaakt met business, user en software requirements. Software requirements zullen we in dit boek nog verder indelen naar functionele software requirements en niet-functionele software requirements. Tekstvak 2.1 bevat de definities die wij als uitgangspunt gebruiken. Figuur 2.1 geeft een overzicht van requirements.

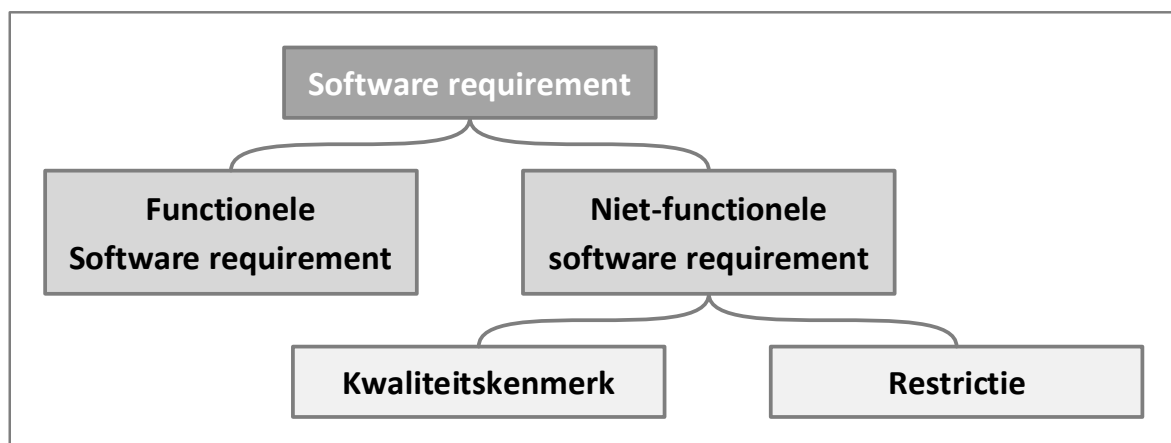
Tekstvak 2.1 Definities van functionele en niet-functionele software requirement

Functionele software requirement

- Een functionele software requirement beschrijft een taak (functie) die een softwareproduct of een systeem moet uitvoeren.

Niet-functionele software requirement

- Een niet-functionele software requirement is een kwaliteitskenmerk en beschrijft hoe goed een softwareproduct of een systeem een taak moet uitvoeren of hoe goed het softwareproduct of het systeem als geheel moet zijn.
- Een niet-functionele software requirement kan ook een restrictie zijn waaraan een softwareproduct of een systeem moet voldoen.



Figuur 2.1 Indeling software requirements

Voorbeeld

1. Functionele software requirement

De geldautomaat moet de rekeninghouder faciliteren om geld op te nemen van de eigen rekening.

2. Kwaliteitskenmerk (niet-functionele software requirement)

De geldautomaat moet de toegang van onbevoegden weigeren tot de rekening van de rechtmatige rekeninghouders.

3. Restrictie (niet-functionele software requirement)

De code voor de kaartlezer moet in de programmeertaal C geschreven zijn.

Toelichting

1. Geld opnemen is een functie van een geldautomaat. Het is een taak die een geldautomaat uitvoert voor de rekeninghouder. Daarom beschouwen we requirement 1 als een functionele software requirement.
2. De requirement 2 gaat over authenticiteit. Is de gebruiker van de bankpas daadwerkelijk de rechtmatige rekeninghouder? Het controleren van authenticiteit beschouwen we als een kwaliteitskenmerk dat een geldautomaat moet bezitten om de rekeninghouder en de andere belanghebbenden te beschermen tegen misbruik. Authenticiteit is volgens ISO 25010 een kwaliteitskenmerk. Daarom is requirement 2 een niet-functionele software requirement. De voorbeelden en opdrachten in dit boek over de kwaliteit van softwareproducten zijn gekoppeld aan ISO 25010. Meer informatie over ISO 25010 vind je in: hoofdstuk 3 ISO 25010 Kwaliteit van softwareproduct.
3. Requirements 3 beschrijft een restrictie (constraint). Een restrictie is een beperking waaraan je gebonden bent.

Oefeningen

Oefening 1

Geef aan welke requirement een functionele en welke een niet-functionele software requirement is. Onderbouw je antwoorden.

ID	Omschrijving requirement
1	De geldautomaat moet saldo tonen aan de rekeninghouder.
2	De geldautomaat moet binnen twee seconden saldo tonen aan de rekeninghouder nadat de rekeninghouder de optie 'saldo tonen' heeft gekozen.
3	De geldautomaat moet altijd het juiste geldbedrag uitkeren (niet te veel, niet te weinig).
4	De geldautomaat moet in de volgende vier talen met de gebruiker kunnen communiceren: Nederlands, Engels, Spaans, Frans.

Oefening 2

De website van de NS is ontworpen, ontwikkeld en getest volgens een reeks functionele en niet-functionele software requirements. Verken de website van de NS en voer vervolgens de volgende twee opdrachten uit.

- 1 Formuleer twee functionele software requirements voor de website van de NS.
- 2 Formuleer twee niet-functionele software requirements voor de website van de NS.

Uitwerkingen van oefeningen

Uitwerking oefening 1

- 1 Functionele software requirement. 'Saldo tonen' is een taak die de geldautomaat uitvoert. Daarom is requirement 1 een functionele requirement.
- 2 Niet-functionele software requirement. Bij requirement 2 staat de responstijd van twee seconden centraal. Het gaat erom hoe snel de geldautomaat de taak 'saldo tonen' uitvoert. Snelheid is volgens ISO 25010 een kwaliteitskenmerk. Daarom is requirement 2 een niet-functionele software requirement.
- 3 Niet-functionele software requirement. Bij requirement 3 staat het juist uitvoeren van de taak 'geld uitkeren' centraal. Het juist uitvoeren van de taak is functionele juistheid. Functionele juistheid is volgens ISO 25010 een kwaliteitskenmerk. Daarom is requirement 3 een niet-functionele software requirement.
- 4 Niet-functionele software requirement. Bij requirement 4 staat de interactie tussen de gebruiker en de geldautomaat centraal. De geldautomaat kan in vier talen met de gebruiker communiceren. De vier talen zorgen ervoor dat een grotere groep de geldautomaat kan gebruiken. Dit heeft te maken met toegankelijkheid. Toegankelijkheid is volgens ISO 25010 een kwaliteitskenmerk. Daarom is requirement 4 een niet-functionele software requirement.

Uitwerking oefening 2

Voor deze oefening kun je talloze uitwerkingen opstellen. De onderstaande uitwerkingen zijn bedoeld als voorbeeld en vergelijkingsmateriaal.

ID	Omschrijving functionele software requirement
1	De website van de NS moet de gebruiker ondersteunen om online een treinkaartje te kopen.
2	De website van de NS moet aan de gebruiker een overzicht geven van de geplande werkzaamheden op het spoor.

ID	Omschrijving niet-functionele software requirement
3	De website van de NS moet voorkomen dat de gebruiker tijdens het kopen van een treinkaartje een datum in het verleden selecteert.
4	De website van de NS moet de gebruiker de mogelijkheid bieden om alle soorten treinkaarten en abonnementen (online) te bestellen die ook bij een fysiek loket te koop zijn.

Toelichting op de uitwerking oefening 2

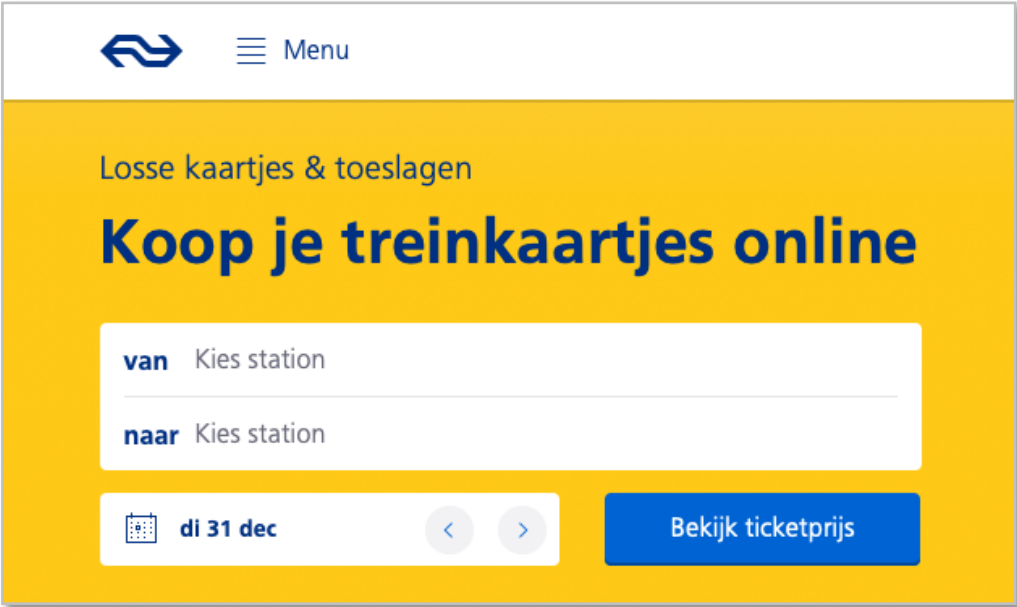
- 1 De requirement met ID 1 omschrijft een taak die de website van de NS uitvoert voor de gebruikers:
 - Een treinkaartje kopen.

‘Een treinkaartje kopen’ is een functionaliteit van de website. Daarom is deze requirement een functionele software requirement. Figuur 2.2 toont een screenshot van deze functionaliteit.

- 2 Requirement 2 is een functionele software requirement omdat deze een functionaliteit beschrijft die de website biedt aan de reizigers en andere belanghebbenden:
 - Een overzicht krijgen van de geplande werkzaamheden op het spoor.
- 3 De intentie van de requirement met ID 3 is om de gebruiker te beschermen tegen het maken van de volgende fout tijdens het kopen van een treinkaart:
 - Een datum in het verleden kiezen.

Het beschermen van de gebruikers tegen het maken van fouten is volgens ISO 25010 een kwaliteitskenmerk (voorkomen gebruikersfouten). Daarom is deze requirement een niet-functionele software requirement. Zie het vakje met de kalender in figuur 2.2.

- 4 Bij requirement 4 staat de volledigheid van functionaliteit centraal. Functionele volledigheid is volgens ISO 25010 een kwaliteitskenmerk. Daarom is deze requirement een niet-functionele software requirement.

The screenshot shows the NS website's interface for purchasing online tickets. At the top, there is a white header with the NS logo and a 'Menu' button. Below the header is a large yellow section with the text 'Losse kaartjes & toeslagen' and a large blue heading 'Koop je treinkaartjes online'. Underneath the heading is a white form with two input fields: 'van Kies station' and 'naar Kies station'. Below these fields is a date selection area showing a calendar icon, the text 'di 31 dec', and left and right navigation arrows. To the right of the date selection is a blue button labeled 'Bekijk ticketprijs'.

Figuur 2.2 Screenshot van online kaartverkoop van de NS

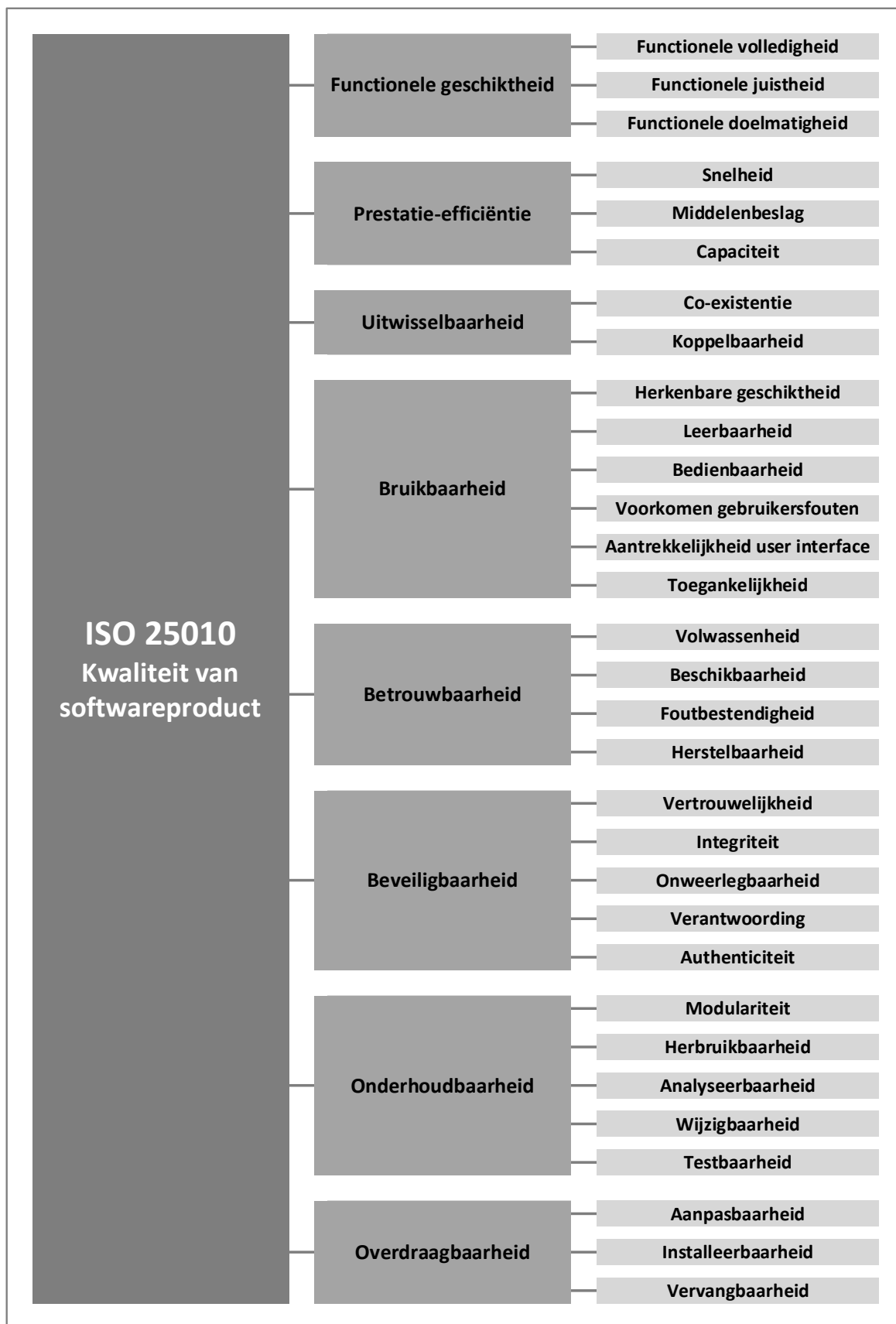
3 ISO 25010 Kwaliteit van softwareproduct

ISO 25010 is een internationale standaard voor het evalueren van de kwaliteit van softwareproducten. Als ICT'er of ICT-student kun je ISO 25010 als referentie gebruiken om de gewenste kwaliteitskenmerken van een softwareproduct te omschrijven (niet-functionele software requirements). De niet-functionele requirements kun je onder andere gebruiken voor het toetsen, testen of meten van de kwaliteit van softwareproducten. Denk bijvoorbeeld aan het opstellen van niet-functionele requirements voor het selecteren van een softwareproduct of een systeem. Figuur 3.1 toont het totale overzicht van de kwaliteitskenmerken van software volgens ISO 25010. Verder in dit hoofdstuk kun je de definities lezen van alle kwaliteitskenmerken met gerelateerde voorbeelden.

Voor het opstellen van de definities hebben we de volgende bronnen gebruikt:

- > <http://www.smartest.nl> (Nederlandstalige brontekst),
- > <https://iso25000.com> (Engelstalige brontekst).

Op Wikipedia kun je ook de volledige definities zowel in het Engels als in het Nederlands vinden. Gebruik hiervoor 'ISO 25010' als zoekterm.



Figuur 3.1 Overzicht ISO 25010 kwaliteit van softwareproduct

Definitie functionele geschiktheid

1	Functionele geschiktheid <i>Functional suitability</i>	De mate waarin een product of systeem functies levert die voldoen aan de uitgesproken en veronderstelde behoeften, bij gebruik onder gespecificeerde condities. Dit kenmerk bestaat uit de volgende subkenmerken:
1.1	Functionele volledigheid <i>Functional completeness</i>	De mate waarin de set van functies alle gespecificeerde taken en gebruikersdoelen afdekt.
1.2	Functionele juistheid <i>Functional correctness</i>	De mate waarin een product of systeem de correcte resultaten met de benodigde mate van nauwkeurigheid levert.
1.3	Functionele doelmatigheid <i>Functional appropriateness</i>	De mate waarin de functies het bereiken van gespecificeerde taken en doelen mogelijk maken.

Voorbeelden

1.1 Functionele volledigheid

De calculator-app moet alle functies bevatten die gespecificeerd zijn in de 'Software Requirements Specification' voor de fysieke rekenmachine HP 35S.

1.2 Functionele juistheid

De infrarood thermometer moet de temperatuur van een persoon met een nauwkeurigheid van ± 0.5 °C meten.

1.3 Functionele doelmatigheid

Het Autonomous Emergency Braking System (AEBS) van de auto moet minimaal 95% van frontale botsingen voorkomen.

Aandachtspunt

Merk op dat de voorbeelden 1.1 t/m 1.3 zich richten op de kwaliteit van de functies. Daarom zijn deze drie requirements niet-functionele requirements.

Hieronder zie je de gerelateerde functionele requirements als voorbeeld:

FR-1.1 De calculator-app moet binaire getallen omzetten naar decimale getallen.

Voor de calculator-app in voorbeeld 1.1 kunnen we in principe tientallen functionele requirements schrijven. We hebben hier ons beperkt tot slechts één functionele requirement te verduidelijking.

FR-1.2 De infrarood thermometer moet de temperatuur van een persoon meten.

FR-1.3 Het Autonomous Emergency Braking System (AEBS) van de auto moet automatisch remmen wanneer de auto dreigt frontaal te botsen met een stilstaand of bewegend object.

Definitie prestatie-efficiëntie

2	Prestatie-efficiëntie <i>Performance efficiency</i>	De prestaties in verhouding tot de hoeveelheid gebruikte middelen onder bepaalde condities. Dit kenmerk bestaat uit de volgende subkenmerken:
2.1	Snelheid <i>Time behaviour</i>	De mate waarin antwoord- en verwerkingstijden en doorvoersnelheid van een product of systeem, tijdens de uitvoer van zijn functies, voldoen aan de eisen.
2.2	Middelenbeslag <i>Resource utilization</i>	De mate waarin de hoeveelheid en soorten middelen die gebruikt worden door een product of systeem, tijdens de uitvoer van zijn functies, voldoen aan de eisen.
2.3	Capaciteit <i>Capacity</i>	De mate waarin de maximale limieten van een product- of systeemparemeter voldoen aan de eisen.

Voorbeelden

2.1 Snelheid

De examen-app moet elke nieuwe vraag binnen 100 ms weergeven nadat de examenkandidaat op de 'Volgende vraag' heeft geklikt.

2.2 Middelenbeslag

De examen-app mag maximaal 50 MB geheugenruimte in beslag nemen op de tablet.

2.3 Capaciteit

Het elektronische examensysteem moet minimaal 1 en maximaal 1000 theorie-examens simultaan kunnen afnemen.

Definitie uitwisselbaarheid

3	Uitwisselbaarheid <i>Compatibility</i>	De mate waarin een product, systeem of component informatie kan uitwisselen met andere producten, systemen of componenten, en/of het de vereiste functies kan uitvoeren terwijl het dezelfde hard- of softwareomgeving deelt. Dit kenmerk bestaat uit de volgende subkenmerken:
3.1	Co-existentie <i>Co-existence</i>	De mate waarin een product zijn gewenste functies efficiënt kan uitvoeren terwijl het een gemeenschappelijke omgeving en middelen deelt met andere producten, zonder nadelige invloed op enig ander product.
3.2	Koppelbaarheid <i>Interoperability</i>	De mate waarin twee of meer systemen, producten of componenten informatie kunnen uitwisselen en de uitgewisselde informatie kunnen gebruiken.

Voorbeelden

3.1 Co-existentie

Het mobiele netwerk moet meer bandbreedte beschikbaar stellen voor regulier dataverkeer op de gebieden waar evenementen plaatsvinden met behoud van de wettelijk vereiste bandbreedte voor het spraakverkeer van en naar hulpdiensten.

3.2 Koppelbaarheid

Het 5G-netwerk moet een bestaand telefoongesprek zonder onderbreking overdragen naar een 4G-netwerk wanneer de beller zich tijdens het gesprek naar een gebied verplaatst waar alleen 4G-netwerk beschikbaar is.

Definitie bruikbaarheid

4	Bruikbaarheid <i>Usability</i>	De mate waarin een product of systeem gebruikt kan worden door gespecificeerde gebruikers om effectief, efficiënt en naar tevredenheid gespecificeerde doelen te bereiken in een gespecificeerde gebruikscontext. Dit kenmerk bestaat uit de volgende subkenmerken:
4.1	Herkenbare geschiktheid <i>Appropriateness recognizability</i>	De mate waarin gebruikers kunnen herkennen of een product of systeem geschikt is voor hun behoeften.
4.2	Leerbaarheid <i>Learnability</i>	<i>Korte vertaling</i> <i>De gebruiker moet het systeem of product kunnen gebruiken en simultaan leren hoe het werkt.</i> <i>Volledige vertaling</i> De mate waarin een product of systeem gebruikt kan worden door gespecificeerde gebruikers om gespecificeerde leerdoelen te bereiken met betrekking tot het gebruik van het product of systeem met effectiviteit, efficiëntie, voldoening en zonder risico, in een gespecificeerde gebruikscontext.
4.3	Bedienbaarheid <i>Operability</i>	De mate waarin een product of systeem attributen heeft die het makkelijk maken om het te bedienen en onder controle te houden.
4.4	Voorkomen gebruikersfouten <i>User error protection</i>	De mate waarin het systeem gebruikers beschermt tegen het maken van fouten.
4.5	Aantrekkelijkheid user interface <i>User interface aesthetics</i>	De mate waarin een user interface het de gebruiker mogelijk maakt om een plezierige en voldoening gevende interactie te hebben.
4.6	Toegankelijkheid <i>Accessibility</i>	De mate waarin een product of systeem gebruikt kan worden door mensen met de meest uiteenlopende eigenschappen en mogelijkheden om een gespecificeerd doel te bereiken in een gespecificeerde gebruikscontext.

Voorbeelden

4.1 Herkenbare geschiktheid

De user interface van de 'Requirements Management Tool' moet een potentiële gebruiker binnen vijf minuten in staat stellen om te beoordelen of de tool als geheel voldoet aan de expliciete en vanzelfsprekende verwachtingen.

4.2 Leerbaarheid

De user interface van de examen-app moet een nieuwe examenkandidaat in staat stellen om de app tijdens het maken van het examen te leren gebruiken zonder het risico te lopen om vragen verkeerd te beantwoorden of tijd te verliezen.

4.3 Bedienbaarheid

De user interface van het navigatiesysteem moet de automobilist de mogelijkheid bieden om tijdens het rijden een bestemming via spraak te wijzigen.

4.4 Voorkomen gebruikersfouten

De user interface van de examen-app moet voorkomen dat een examenkandidaat een examen inlevert voor beoordeling terwijl één of meer meerkeuzevragen per ongeluk onbeantwoord zijn gebleven.

4.5 Aantrekkelijkheid user interface

De user interface van de elektronische leeromgeving moet ervoor zorgen dat de cursisten op een plezierige manier de leeromgeving kunnen gebruiken.

4.6 Toegankelijkheid

De user interface van de afsprakenplanner van het gezondheidscentrum moet ervoor zorgen dat een gebruiker met visuele beperkingen zelf een afspraak kan plannen met de arts.

Definitie betrouwbaarheid

5	Betrouwbaarheid <i>Reliability</i>	De mate waarin een systeem, product of component gespecificeerde functies uitvoert onder gespecificeerde condities gedurende een gespecificeerde hoeveelheid tijd. Dit kenmerk bestaat uit de volgende subkenmerken:
5.1	Volwassenheid <i>Maturity</i>	De mate waarin een systeem, product of component aan betrouwbaarheidsbehoeften voldoet onder normale werkomstandigheden.
5.2	Beschikbaarheid <i>Availability</i>	De mate waarin een systeem, product of component operationeel en toegankelijk is wanneer men het wil gebruiken.
5.3	Foutbestendigheid <i>Fault tolerance</i>	De mate waarin een systeem, product of component werkt zoals bedoeld ondanks de aanwezigheid van hardware- of softwarefouten.
5.4	Herstelbaarheid <i>Recoverability</i>	De mate waarin het product of systeem, in geval van een onderbreking of bij een fout, de direct betrokken gegevens kan herstellen en het systeem in de gewenste staat kan terugbrengen.

Voorbeelden

5.1 Volwassenheid

Het veiligheidssysteem moet in 99% van gevallen juist aangeven of een persoon op de bouwplaats een veiligheidshelm draagt.

5.2 Beschikbaarheid

Het mobiele netwerk mag voor gepland onderhoud maximaal een keer per maand en per keer maximaal vijf minuten en alleen tussen 4:00 en 4:30 uur in de ochtend voor gepland onderhoud onbereikbaar zijn.

5.3 Foutbestendigheid

Het mobiele netwerk moet alle communicatiediensten onafgebroken voortzetten wanneer tot 50% van de interconnectiepunten uitvalt.

5.4 Herstelbaarheid

De Gateway van het mobiele netwerk moet na een stroomuitval zichzelf automatisch terugbrengen naar de oorspronkelijke status.

Definitie beveiligbaarheid

6	Beveiligbaarheid <i>Security</i>	De mate waarin een product of systeem informatie en gegevens beschermt zodat personen, andere producten of systemen de juiste mate van gegevenstoegang hebben, passend bij hun soort en niveau van autorisatie. Dit kenmerk bestaat uit de volgende subkenmerken:
6.1	Vertrouwelijkheid <i>Confidentiality</i>	De mate waarin een product of systeem ervoor zorgt dat gegevens alleen toegankelijk zijn voor degenen die geautoriseerd zijn.
6.2	Integriteit <i>Integrity</i>	De mate waarin een systeem, product of component ongeautoriseerde toegang tot of aanpassing van computerprogramma's of gegevens verhindert.
6.3	Onweerlegbaarheid <i>Non-repudiation</i>	De mate waarin kan worden bewezen dat acties of gebeurtenissen hebben plaatsgevonden, zodat later deze acties of gebeurtenissen niet ontkend kunnen worden.
6.4	Verantwoording <i>Accountability</i>	De mate waarin acties van een entiteit getraceerd kunnen worden naar die specifieke entiteit.
6.5	Authenticiteit <i>Authenticity</i>	De mate waarin bewezen kan worden dat de identiteit van een onderwerp of bron is zoals wordt beweerd.

Voorbeelden

6.1 Vertrouwelijkheid

Het patiëntenregistratiesysteem moet alleen geautoriseerde artsen en medewerkers toegang geven tot de medische gegevens van de patiënten.

6.2 Integriteit

Het patiëntenregistratiesysteem moet alleen geautoriseerde artsen en medewerkers toestaan om medische gegevens van de patiënten in te voeren, aan te passen of te verwijderen.

6.3 Onweerlegbaarheid

Het patiëntenregistratiesysteem moet alle handelingen die gebruikers in het systeem verrichten vastleggen en bewaren volgens de in de wet vastgestelde periode.

6.4 Verantwoording

Het patiëntenregistratiesysteem moet alle uitgewisselde documenten tussen elke medewerker en elke patiënt bewaren volgens de in de wet vastgestelde periode.

6.5 Authenticiteit

Het patiëntenregistratiesysteem moet de echtheid van de identiteit van de gebruiker vaststellen, zodat alleen de bevoegde gebruikers toegang krijgen tot het systeem.

Definitie onderhoudbaarheid

7	Onderhoudbaarheid <i>Maintainability</i>	De mate van effectiviteit en efficiëntie waarmee een product of systeem gewijzigd kan worden om het te verbeteren, te corrigeren of aan te passen aan veranderingen in de omgeving en in de behoeften. Dit kenmerk bestaat uit de volgende subkenmerken:
7.1	Modulariteit <i>Modularity</i>	De mate waarin een systeem of computerprogramma opgebouwd is in losstaande componenten, zodat wijzigingen van een component minimale impact hebben op andere componenten.
7.2	Herbruikbaarheid <i>Reusability</i>	De mate waarin een bestaand onderdeel gebruikt kan worden in meer dan één systeem of bij het bouwen van een nieuw onderdeel.
7.3	Analyseerbaarheid	De mate van effectiviteit en efficiëntie waarmee het mogelijk is om: 1) de impact te beoordelen van een geplande verandering van één of meer onderdelen van een product of systeem, 2) tekortkomingen en/of foutoorzaken van een product vast te stellen, 3) onderdelen te identificeren die gewijzigd moeten worden.
7.4	Wijzigbaarheid <i>Modifiability</i>	De mate waarin een product of systeem effectief en efficiënt kan worden aangepast zonder defecten te veroorzaken of bestaande productkwaliteit te verslechteren.
7.5	Testbaarheid <i>Testability</i>	De mate waarin effectief en efficiënt testcriteria vastgesteld kunnen worden voor een systeem, product of component en waarin tests uitgevoerd kunnen worden om vast te stellen of aan die criteria is voldaan.

Voorbeelden

7.1 Modulariteit

Het betaalsysteem van de webshop moet modulair zijn opgebouwd en elke module moet slechts één taak uitvoeren.

7.2 Herbruikbaarheid

Het betaalsysteem van de webshop moet ook geschikt zijn als betaalsysteem voor gebruik bij andere diensten.

7.3 Analyseerbaarheid

De add/drop multiplexer moet een diagnoserapport over de eigen status versturen naar het 'Network Management System', wanneer de multiplexer abnormale waarden detecteert op de meetpunten van de hardware.

7.4 Wijzigbaarheid

De add/drop multiplexer moet herconfigureerbaar zijn, zodat de network engineers verbindingen kunnen toevoegen, veranderen en verwijderen.

7.5 Testbaarheid

Elk netwerkelement moet in de productiefase de eigen hard- en software volgens de testcriteria en -routines van de fabrikant kunnen testen.

Definitie overdraagbaarheid

8	Overdraagbaarheid	De mate waarin een systeem, product of component effectief en efficiënt overgezet kan worden van één hardware, software of andere operationele of gebruiksomgeving naar een andere. Dit kenmerk bestaat uit de volgende subkenmerken:
8.1	Aanpasbaarheid <i>Adaptability</i>	De mate waarin een product of systeem effectief en efficiënt aangepast kan worden: 1) voor verschillende hardware, software of andere operationele of gebruiksomgevingen; 2) voor zich ontwikkelende hardware, software of andere operationele of gebruiksomgevingen.
8.2	Installeerbaarheid <i>Installability</i>	De mate waarin het product of het systeem effectief en efficiënt geïnstalleerd of verwijderd kan worden in een gespecificeerde omgeving.
8.3	Vervangbaarheid <i>Replaceability</i>	De mate waarin een product een ander gespecificeerd softwareproduct voor hetzelfde doel in dezelfde omgeving kan vervangen.

Voorbeelden

8.1 Aanpasbaarheid

De calculator-app moet van iOS naar Android kunnen worden overgezet.

8.2 Installeerbaarheid

Elke nieuwe versie van de antivirussoftware moet zich in de bestaande omgeving volledig automatisch kunnen installeren.

8.3 Vervangbaarheid

Versie 4.5 van het ingebouwde navigatiesysteem van de auto moet op dezelfde hardware en hetzelfde besturingssysteem kunnen opereren als versie 4.4.

Oefeningen

In dit hoofdstuk heb je kennis gemaakt met kwaliteitskenmerken die in ISO 25010 zijn gedefinieerd. De volgende oefeningen geven je meer inzicht in waarom en hoe je ISO 25010 in de praktijk kunt gebruiken voor het waarborgen en aantonen van softwarekwaliteit.

Oefening 1

Voor een product zijn er ruim tweehonderd niet-functionele requirements opgesteld. Je krijgt van je projectleider de opdracht om deze requirements te classificeren en vervolgens in aparte documenten te clusteren volgens de kwaliteitskenmerken die in ISO 25010 zijn gedefinieerd. De projectleider weet dat deze opdracht je veel tijd gaat kosten terwijl je deze tijd hard nodig hebt voor het behalen van andere deadlines. Waarom wil de projectleider de niet-functionele requirements in aparte documenten clusteren?

Oefening 2

Het aantoonbaar maken van de kwaliteit van een softwareproduct of een systeem in een afstudeerproject is een uitdaging voor studenten. Beschrijf hoe je ISO 25010 als referentie kunt gebruiken om de kwaliteit van het door jou ontwikkelde softwareproduct of systeem aan te tonen bij je docenten en andere betrokkenen die invloed hebben op je eindcijfer in je afstudeerproject.

Oefening 3

Beschrijf hoe je in een afstudeerproject ISO 25010 als checklist kunt gebruiken voor het vaststellen van prioriteiten voor het testen van je softwareproduct of systeem.

Uitwerkingen van oefeningen

Uitwerking oefening 1

Figuur 3.1 laat zien dat een softwareproduct verschillende soorten kwaliteitskenmerken heeft. Het realiseren van deze kwaliteitskenmerken kan zeer specialistisch werk zijn dat je moet uitbesteden aan verschillende experts of zelfs aan verschillende bedrijven. Een security tester kan en wil waarschijnlijk geen usability testen uitvoeren. Security en usability testen zijn verschillende specialisaties. Hiervoor heb je ook verschillende tools, testomgevingen en specialisten nodig. Daarom wil de projectleider de niet-functionele requirements clusteren in aparte documenten om deze als formele opdrachten te kunnen uitbesteden aan verschillende bedrijven of experts.

Uitwerking oefening 2

Je bent een ICT-student en je bent bezig met je afstudeerproject. Binnen je project ga je een nieuw subsysteem ontwikkelen en toevoegen aan een bestaand systeem. Je onderzoekt onder andere de huidige architectuur, de software requirements en de passende technologieën voor het realiseren van het subsysteem.

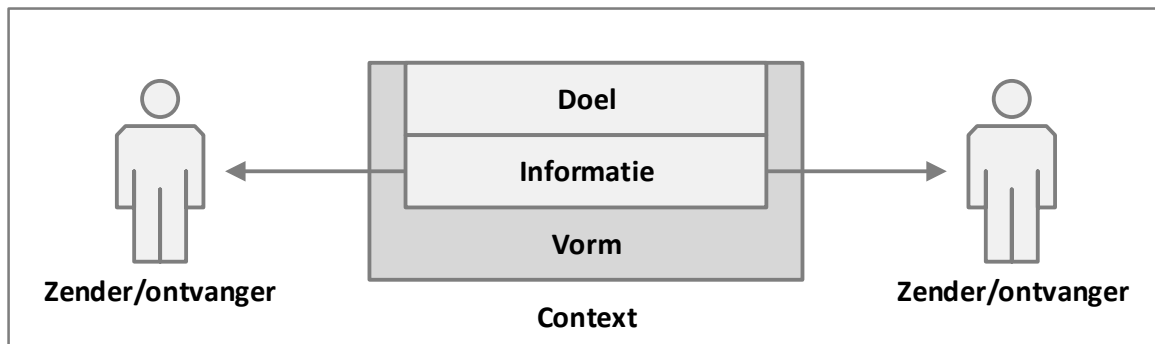
Je gebruikt ISO 25010 als referentie om niet-functionele requirements te schrijven. Je gebruikt deze requirements als uitgangspunt voor het waarborgen van de kwaliteit van het (sub)systeem in het hele ontwikkelproject. De requirements voeg je toe aan je afstudeerdocumentatie. Tijdens de eindpresentatie of de verdediging van je afstudeerproject ga je de meest relevante requirements gebruiken om aan te tonen dat je het juiste product op de juiste manier hebt ontwikkeld voor je opdrachtgever. De docenten en de andere betrokkenen bij je afstudeerproject kunnen de requirements als extra criteria gebruiken om je werk gericht te waarderen.

Uitwerking oefening 3

Een afstudeerproject duurt normaal 20 weken (een semester). Het testen van het softwareproduct is een van de taken waarop je prestatie wordt beoordeeld. Je kunt niet alles testen binnen de beperkte tijd die je hebt. Daarom moet je je richten op de meest cruciale kwaliteitskenmerken. ISO 25010 is een handig communicatiemiddel om samen met de opdrachtgever en docenten de prioriteiten te bepalen die voor de opleiding en voor de opdrachtgever cruciaal zijn voor de eindbeoordeling.

4 Handreikingen voor requirements schrijven

Zonder informatie kun je geen software ontwikkelen voor en met anderen. Er zijn verschillende vormen om informatie uit te wisselen met andere betrokkenen in het ontwikkelproces van software. Denk bijvoorbeeld aan vormen zoals mondelinge communicatie (face-to-face), videogesprekken, schriftelijke communicatie en animaties. Welke vorm van communicatie je kiest, is afhankelijk van het doel, de doelgroep, het type informatie en de context. Figuur 4.1 toont een conceptueel model voor het uitwisselen van informatie waarin de bovengenoemde aspecten zijn weergegeven.



Figuur 4.1 Conceptueel model voor informatie uitwisselen

In dit boek tref je veertien handreikingen aan voor het schrijven van requirements. Tabel 4.1 geeft een overzicht van deze handreikingen. Elke handreiking is voorzien van een ID. De ID's zijn handig bij het geven van feedback aan de schrijvers van requirements. Op deze manier kan de schrijver van de requirements in dit boek lezen wat de mogelijke verbeteracties zijn.

Handreikingen zijn slechts hulpmiddelen

Merk op dat deze handreikingen zijn bedoeld als hulpmiddel. Je kunt ze niet standaard in elke situatie (even zwaar) toepassen. Je wilt bijvoorbeeld minder gedetailleerde requirements schrijven voor wegwerpsoftware. Bij een openbare aanbesteding waarbij je een softwareleverancier wilt selecteren voor een overheidsinstelling, wil je de requirements als selectiecriteria gebruiken. Deze ga je volledig, logisch, nauwkeurig, eenduidig, traceerbaar, verifieerbaar en sorteerbaar schrijven. Je wilt je keuzes onderbouwen als belanghebbenden en vooral de afgewezen leveranciers vragen stellen over het selectieproces.

Prioriteiten voor requirements

In dit boek hanteren we drie prioriteiten voor requirements volgens ISO IEEE 830-1998:

- 1 Verplicht (*Mandatory*)
- 2 Optioneel (*Optional*)
- 3 Aangenaam (*Nice-to-have*).

Een requirement is:

- 1 Verplicht als deze onmisbaar is voor het succes van het systeem.
- 2 Optioneel als deze zonder risico's kan worden gemist.

- 3 Aangenaam als deze geen invloed heeft op het succes van het systeem, maar wel is leuk om te hebben.

Je begint met het realiseren van verplichte requirements. Wanneer je deze hebt gerealiseerd, ga je verder met optionele en vervolgens met aangename requirements.

MoSCoW-methode als alternatief voor ISO IEEE 830-1998

De MoSCoW-methode is ook een wijze van prioriteiten stellen. De prioriteiten zijn ingedeeld in vier niveaus:

Een requirement krijgt de prioriteit:

- 1 **M** – *Must have* als deze onmisbaar is voor het succes van het systeem.
- 2 **S** – *Should have* als deze zonder risico's kan worden gemist.
- 3 **C** – *Could have* als deze geen invloed heeft op het succes van het systeem, maar wel aangenaam is om te hebben.
- 4 **W** – *won't have* als deze op dit moment niet nuttig is maar wel interessant kan zijn op een later tijdstip.

De kleine letters 'o' in MoSCoW hebben geen betekenis, maar maken de afkorting makkelijker te onthouden.

Tabel 4.1 *Overzicht handreikingen voor requirements schrijven*

ID	Handreiking
H01.	Beschrijf slechts één behoefte, eis of restrictie per requirement.
H02.	Formuleer eenduidige requirements.
H03.	Formuleer elke requirement precies en volledig.
H04.	Gebruik concrete woorden voor inzichtelijkheid.
H05.	Gebruik abstracte woorden voor overzichtelijkheid.
H06.	Wees consequent in je woordkeuze.
H07.	Formuleer requirements volgens een vaste structuur.
H08.	Formuleer requirements met actieve zinnen.
H09.	Formuleer requirements positief.
H10.	Maak requirements traceerbaar.
H11.	Schrijf requirements die realiseerbaar zijn.
H12.	Maak requirements verifieerbaar.
H13.	Stel een verklarende woordenlijst op voor requirements.
H14.	Verrijk requirements met visuele modellen.

H01. Beschrijf slechts één behoefte, eis of restrictie per requirement

Het is belangrijk dat je slechts één behoefte, eis of restrictie per requirement beschrijft, zodat je elke requirement afzonderlijk kunt sorteren en behandelen in een project.

Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	De bank-app moet de rechtmatige rekeninghouder toegang geven tot de eigen rekening op basis van gezichtsherkenning, vervolgens binnen 100 ms de startpagina tonen en een melding geven van nieuwe features.	Onbepaald	Onbepaald

De requirement in het voorbeeld bevat drie verschillende eisen: authenticiteit, snelheid en functionaliteit. Authenticiteit valt onder beveiligbaarheid; snelheid valt onder prestatie-efficiëntie. Deze zijn kwaliteitskenmerken. Je kunt ervan uitgaan dat je voor het realiseren van deze twee kenmerken twee verschillende specialisten en technische middelen nodig hebt. Specialistische taken wil je soms uitbesteden aan andere partijen. Deze partijen kunnen van verschillende bedrijven zijn. Daarnaast kunnen de prioriteiten per kenmerk verschillend zijn. Dit kan betekenen dat je ze op verschillende momenten wilt plannen. Daarom moet je deze drie eisen splitsen, zodat je ze apart kunt behandelen in het ontwikkelproces.

Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	De bank-app moet de rechtmatige rekeninghouder toegang geven tot de eigen rekening op basis van gezichtsherkenning.	Authenticiteit	Verplicht
2.	De bank-app moet de startpagina binnen 100 ms weergeven nadat de app het inlogproces heeft afgerond.	Snelheid	Verplicht
3.	De bank-app moet op de startpagina een melding weergeven over nieuwe features zodra de app het inlogproces heeft afgerond.	Functionaliteit	Optioneel

H02: Formuleer eenduidige requirements

Requirements moeten eenduidig zijn, zodat de ICT'ers de juiste software op de juiste manier ontwikkelen voor de belanghebbenden. Lezers en vooral schrijvers zijn zich er niet altijd van bewust dat een tekst voor meer dan één uitleg vatbaar is. Daarom is het verstandig om de requirements door verschillende mensen te laten toetsen. Face-to-face communiceren over requirements is natuurlijk ook een krachtige manier om zeker te stellen dat alle betrokkenen de requirements op dezelfde manier opvatten.

☹ Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1	Het factureringssysteem moet de data op twee verschillende opslagruimtes bewaren.	Onbepaald	Verplicht

De requirement in het voorbeeld kun je op meerdere manieren opvatten. We geven hier twee mogelijke interpretaties:

1. Het factureringssysteem begint met het opslaan van de data in opslagruimte 1. Wanneer opslagruimte 1 vol raakt, gaat het systeem verder met opslagruimte 2. De data in opslagruimte 1 en 2 zijn niet identiek.
2. Het factureringssysteem bewaart exact dezelfde data als geheel op twee verschillende opslagruimtes. De gehele data op de twee opslagruimtes zijn altijd identiek. Een opslagruimte is standaard actief. Een opslagruimte fungeert standaard als backup.

We geven alleen voor de tweede interpretatie een beter alternatief.

😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het factureringssysteem moet altijd exact dezelfde data als geheel op twee gescheiden fysieke opslagruimtes bewaren. <i>Toelichting</i> Opslagruimte 1 is standaard actief. Opslagruimte 2 fungeert standaard als backup.	Herstelbaarheid	Verplicht
2.	De actieve en backup opslagruimtes voor de data moeten zich op twee gescheiden locaties bevinden met een rijafstand van minimaal 20 km en maximaal 50 km van elkaar.	Herstelbaarheid	Verplicht

H03: Formuleer elke requirement precies en volledig

Een requirement beschouwen we hier als precies en volledig als deze alle relevante informatie op een nauwkeurige wijze beschrijft voor de beoogde lezers.

☹ Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het elektronische toetssysteem moet aan minimaal 50 studenten toegang geven tot een officiële toets, zodat ze de toets kunnen maken.	Capaciteit	Verplicht

In de requirement in het voorbeeld is niet aangegeven wat het minimum- en maximaantal studenten moet zijn dat individueel of in groepsverband simultaan een

toets maakt. De requirement is onnauwkeurig en onvolledig. Hierdoor kun je als ontwerper, programmeur of tester de limieten van het systeem niet bepalen en testen. Daarnaast kun je geen capaciteiten huren/kopen als je gebruik wilt maken van clouddiensten.

😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het elektronische toetssysteem moet voor elke officiële toetssessie aan minimaal 1 student en maximaal 500 studenten toegang geven tot de toets.	Capaciteit	Verplicht

H04: Gebruik concrete woorden voor inzichtelijkheid

Requirements moeten concreet zijn wanneer ICT'ers voor hun ontwerp, code en test gedetailleerde informatie nodig hebben. In het volgende voorbeeld zijn de woorden 'gebruiker' en 'verwerken' te abstract. Als ICT'er moet je deze woorden concreter maken. Dit kun je doen door verhelderende vragen te stellen aan de belanghebbenden die de requirements aanleveren. Wie zijn de gebruikers? Wat houdt cijfer verwerken in?

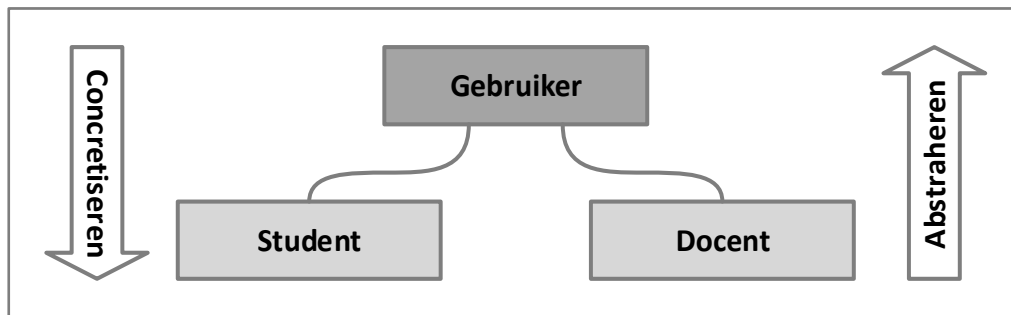
😞 Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	Gebruiker wil cijfer lezen.	Functionaliteit	Verplicht
2.	Gebruiker wil cijfer verwerken.	Functionaliteit	Verplicht

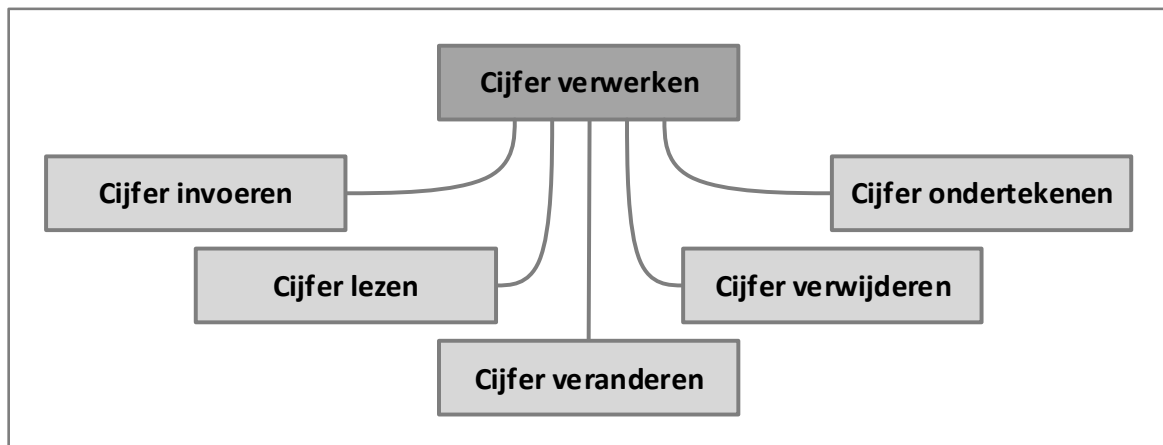
😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Student wil eigen cijfer lezen.	Functionaliteit	Verplicht
2.	Docent wil cijfer verwerken.	Functionaliteit	Verplicht
2.1	Docent wil cijfer invoeren.	Functionaliteit	Verplicht
2.2	Docent wil cijfer lezen.	Functionaliteit	Verplicht
2.3	Docent wil cijfer veranderen.	Functionaliteit	Verplicht
2.4	Docent wil cijfer verwijderen.	Functionaliteit	Verplicht
2.5	Docent wil cijfer ondertekenen.	Functionaliteit	Verplicht

Figuur 4.2 en 4.3 laten de verbanden zien tussen abstracte en concrete woorden. Een abstract woord kun je concretiseren door het woord te splitsen in specifiekere woorden die binnen dezelfde verzameling blijven. Concrete woorden kun je abstraheren door deze te laten representeren door een generiek woord dat de concrete woorden als een verzamelnaam weergeeft. Merk op dat je in het ontwikkelproces van software zowel abstracte woorden als concrete woorden nodig hebt om informatie toereikend te maken voor de beoogde doelgroepen en doelen.

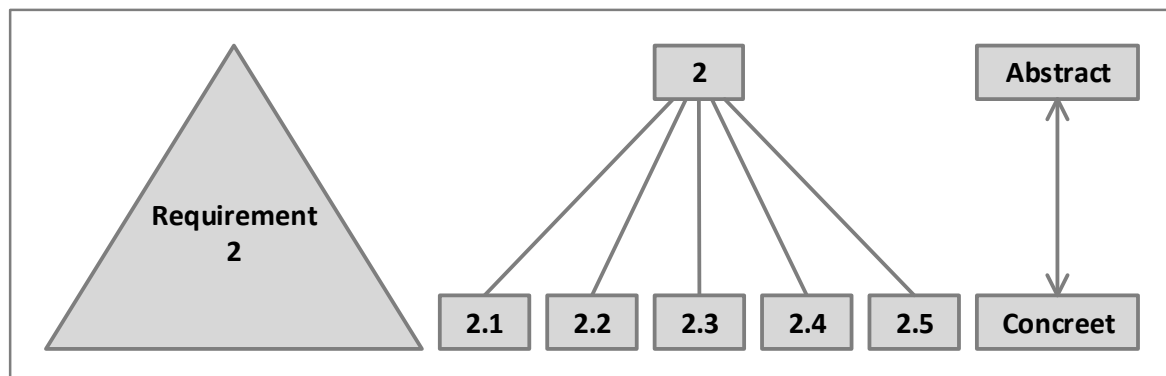


Figuur 4.2 Concretiseren en abstraheren



Figuur 4.3 Cijfer verwerken concretiseren

Figuur 4.4 toont de logische structuur van requirement 2. Tabel 4.2 laat de relatie tussen de taken en de actoren zien. Deze zijn afgeleid van de user requirements.



Figuur 4.4 Structuur requirement 2 'cijfer verwerken'

Tabel 4.2 Relatie taken en actoren

	Actor	Student	Docent
Taak			
Cijfer invoeren		Nee	Ja
Cijfer lezen		Ja	Ja
Cijfer veranderen		Nee	Ja

Cijfer verwijderen	Nee	Ja
Cijfer ondertekenen	Nee	Ja

H05: Gebruik abstracte woorden voor overzichtelijkheid

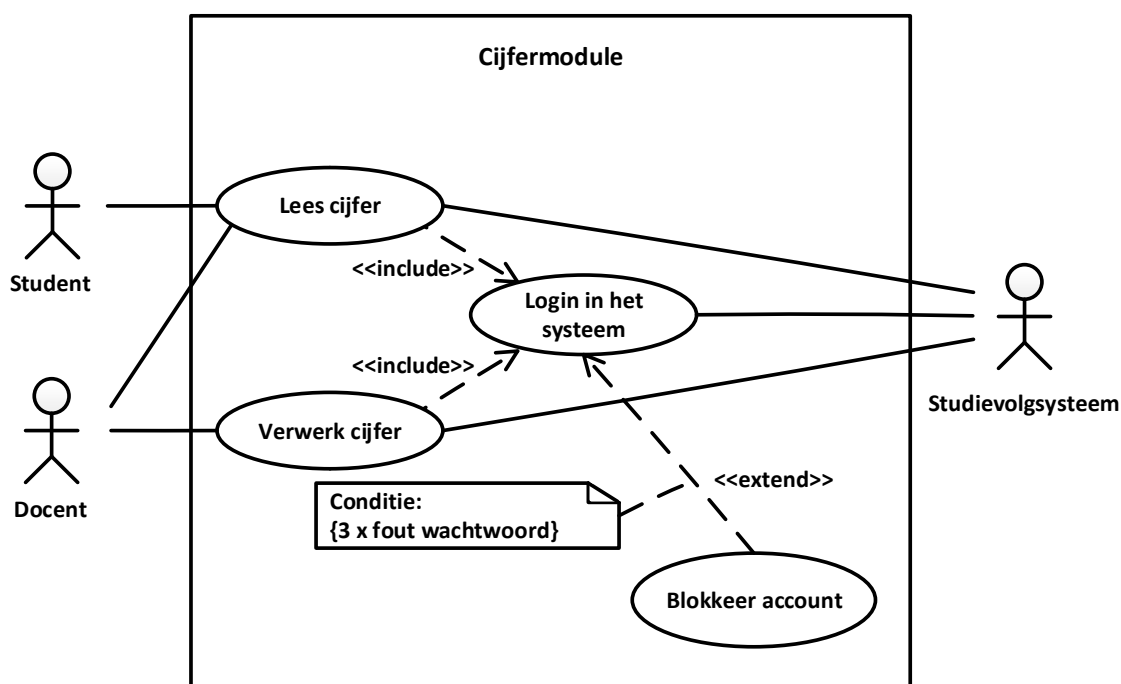
Concrete woorden kunnen soms de overzichtelijkheid van informatie verminderen voor bepaalde doelgroepen en doelen. In soortgelijke situaties wil je abstracte woorden gebruiken. Denk bijvoorbeeld aan het schrijven van globale requirements of het maken van een globaal ontwerp ('global design').

Soms kun je geen concrete woorden gebruiken omdat je op dat moment nog te weinig specifieke informatie hebt. Dat is geen probleem. In het ontwikkelproces van software hebben de betrokkenen tijd nodig om abstracte informatie te concretiseren. Abstracte woorden zijn dan je startpunten om steeds specifieker te worden.

😊 Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	Student wil eigen cijfer lezen.	Functionaliteit	Verplicht
2.	Docent wil cijfer verwerken.	Functionaliteit	Verplicht

Figuur 4.5 laat een use case diagram zien met vier use cases: Lees cijfer, verwerk cijfer, log in het systeem en blokkeer account. Merk op dat het diagram te onoverzichtelijk zou worden als we de use case 'verwerk cijfer' hadden geconcretiseerd en vervangen door: voer cijfer in, lees cijfer, verander cijfer, verwijder cijfer, onderteken cijfer.



Figuur 4.5 Use case diagram cijfermodule

H06: Wees consequent in je woordkeuze

Bij het ontwerpen en schrijven van onderhoudbare code werk je doordacht en consistent als het gaat om de naamgeving voor variabelen, constanten, methodes enzovoort. Deze werkwijze moet je ook hanteren voor het schrijven van requirements. Pas alleen woordvariaties toe als het noodzakelijk is. Gebruik voor elk subject consequent dezelfde naam.

☹ Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	Gebruiker wil een product bestellen.	Functionaliteit	Verplicht
2.	Bezoeker wil een product terugsturen.	Functionaliteit	Verplicht
3.	Klant wil een wensenlijst opstellen.	Functionaliteit	Optioneel

😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Geregistreerde klant wil een product bestellen.	Functionaliteit	Verplicht
2.	Geregistreerde klant wil een product terugsturen.	Functionaliteit	Verplicht
3.	Geregistreerde klant wil een wensenlijst opstellen.	Functionaliteit	Optioneel

Er zijn situaties waarbij het beter is om verschillende namen te gebruiken voor hetzelfde subject. Denk bijvoorbeeld aan een gebruiker die een kamer wil reserveren op de website van een hotel. Op het moment dat deze gebruiker zich bij de receptie van het hotel aanmeldt, wil je hem of haar geen gebruiker noemen. In andere systemen van het hotel wordt de gebruiker van de website een hotelgast.

☹ Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	Gebruiker van de website van het hotel wil een kamer reserveren.	Functionaliteit	Verplicht
2.	De receptioniste wil de gebruiker inchecken.	Functionaliteit	Verplicht

😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Gebruiker van de website van het hotel wil een kamer reserveren.	Functionaliteit	Verplicht
2.	De receptioniste wil de hotelgast inchecken.	Functionaliteit	Verplicht

H07: Formuleer requirements volgens een vaste structuur

Technische teksten zoals requirements moet je volgens een vaste en eenvoudige structuur formuleren, zodat de lezers zich op de inhoud kunnen concentreren. Daarnaast maakt een vaste structuur het schrijfproces en de samenwerking efficiënter als meerdere specialisten gelijktijdig requirements schrijven voor een systeem. Begin je requirement altijd met de naam van een belanghebbende of de naam van een (deel)systeem gevolgd door een werkwoord.

Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	Volgens de network architect moet het beheersysteem de operationele status van 1000 routers simultaan kunnen bewaken.	Capaciteit	Verplicht
2.	Er wordt een foutcode weergegeven als een router abnormaal gedrag vertoont.	Analyseerbaarheid	Verplicht

Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het beheersysteem moet de status van 1000 routers simultaan kunnen bewaken.	Capaciteit	Verplicht
2.	Het beheersysteem moet een foutcode weergeven als een router abnormaal gedrag vertoont.	Analyseerbaarheid	Verplicht

H08: Formuleer actieve zinnen

In een actieve zin is het duidelijk wie of wat de actor is en welke handelingen deze actor verricht. Een actor kan een persoon, een instantie of een component zijn die een handeling kan verrichten. Actieve zinnen in requirements kun je makkelijker vertalen naar ontwerpen, algoritmes, broncodes en testgevallen omdat bij deze zinnen de actoren en de handelingen bekend zijn.

Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	De positie van defecte hardware moet worden weergegeven.	Analyseerbaarheid	Verplicht
2.	Elk alarm moet worden bijgewerkt.	Bedienbaarheid	Verplicht

Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het beheersysteem moet de positie van defecte hardware schematisch weergeven.	Analyseerbaarheid	Verplicht

2.	Het beheersysteem moet de network engineer de mogelijkheid bieden om de status van elk alarm handmatig te wijzigen.	Bedienbaarheid	Verplicht
----	---	----------------	-----------

H09: Formuleer requirements positief

Positieve zinnen in requirements zijn eenvoudiger om te zetten naar een ontwerp, code of test dan negatieve zinnen. Vooral dubbele ontkenningen maken de kans op fouten groter.

☹ Voorbeeld 1

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het patiëntenregistratiesysteem moet geen toegang geven tot het systeem aan gebruikers die niet bevoegd zijn.	Authenticiteit	Verplicht

😊 Beter alternatief A

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het patiëntenregistratiesysteem moet alleen de bevoegde gebruikers toegang geven tot het systeem.	Authenticiteit	Verplicht

😊 Beter alternatief B

ID	Omschrijving requirement	Soort	Prioriteit
1.	Het patiëntenregistratiesysteem moet de toegang van ongeautoriseerde gebruikers tot het systeem tegenhouden.	Authenticiteit	Verplicht

☹ Voorbeeld 2

ID	Omschrijving requirement	Soort	Prioriteit
2.	Het patiëntenregistratiesysteem moet ervoor zorgen dat geen ongeautoriseerde gebruikers toegang krijgen tot de medische dossiers van de patiënten.	Vertrouwelijkheid	Verplicht

😊 Beter alternatief A

ID	Omschrijving requirement	Soort	Prioriteit
2.	Het patiëntenregistratiesysteem moet ervoor zorgen dat alleen de geautoriseerde gebruikers toegang krijgen tot de medische dossiers van de patiënten.	Vertrouwelijkheid	Verplicht

😊 Beter alternatief B

ID	Omschrijving requirement	Soort	Prioriteit
2.	Het patiëntenregistratiesysteem moet de toegang van ongeautoriseerde gebruikers tot de medische dossiers van de patiënten tegenhouden.	Vertrouwelijkheid	Verplicht

H10: Maak requirements traceerbaar

Een requirement is traceerbaar als de lezer weet wat de voorgaande en voorliggende documenten zijn en waar ze te vinden zijn.

Voorbeeld

In een ontwikkelproject leiden testers testgevallen af van requirements. Elke requirement heeft een unieke ID (identifier) waardoor de betrokkenen de requirements gericht kunnen zoeken. Elk testgeval heeft ook een ID. De ID bestaat uit twee delen. Het eerste deel heeft exact dezelfde code als de requirement waarvan het testgeval is afgeleid. Het tweede deel is uniek voor het testgeval zelf:

- FR1-TG1
- FR1-TG2

FR1 is de label voor de requirement. TG1 en TG2 zijn de labels voor de testgevallen. Deze maken de requirements en de testgevallen traceerbaar.

Als requirements in een document gebundeld zijn, kun je een lijst opstellen van de gerelateerde documenten.

H11: Schrijf requirements die realiseerbaar zijn

Realiseerbaarheid van een requirement kan te maken hebben met diverse factoren. Denk bijvoorbeeld aan technologie, kennis, tijd en geld. Je kunt bijvoorbeeld op dit moment geen robots ontwikkelen die verpleegkundigen en artsen volledig kunnen vervangen. Dat is niet realiseerbaar. Je kunt wel robots ontwikkelen die verpleegkundigen en artsen kunnen ondersteunen. Onderzoek in hoeverre elke requirement realiseerbaar is voordat je deze daadwerkelijk in het ontwikkelproces opneemt.

H12: Maak requirements verifieerbaar

Een requirement is verifieerbaar als het mogelijk is om op basis van de requirement het product te meten, te toetsen of te testen.

😞 Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
1.	De user interface van de webshop moet gebruikersfouten voorkomen.	Voorkomen gebruikersfouten	Verplicht

😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
1.	De user interface van de webshop moet voorkomen dat de gebruiker een negatief getal bij het aantal producten invoert.	Voorkomen gebruikersfouten	Verplicht

😞 Voorbeeld

ID	Omschrijving requirement	Soort	Prioriteit
2.	De laptop moet snel opstarten.	Snelheid	Verplicht

😊 Beter alternatief

ID	Omschrijving requirement	Soort	Prioriteit
2.	De laptop moet vanuit de toestand 'uit' binnen 40 seconden het startscherm tonen nadat de gebruiker één keer de aan-uitschakelaar heeft ingedrukt.	Snelheid	Verplicht

H13: Stel een verklarende woordenlijst op voor requirements

Het is belangrijk dat je in de documentatie van de requirements een verklarende woordenlijst opneemt, zodat je zekerstelt dat alle betrokkenen de technische en domeinspecifieke termen op dezelfde manier interpreteren. Je vermindert hiermee de kans op fouten. Daarnaast maak je je documentatie gebruikersvriendelijker.

H14: Verrijk requirements met visuele modellen

Visuele modellen kunnen de kwaliteit van informatie en communicatie verhogen als het gaat om requirements. Denk bijvoorbeeld aan: wire frames, user story maps, use case diagrams, activity diagrams, class diagrams, sequence diagrams en state machine diagrams. Het behandelen van deze modellen valt buiten het bestek van dit boek. We raden de volgende twee boeken aan als voorbeeld voor zelfstudie:

1. Laan, G.J. (2007). *Softwareontwikkeling met UML en Java*. Amsterdam: Pearson Education Benelux.
2. Beatty, J. & Chen, A. (2012). *Visual models for software requirements*. Redmond, WA: Microsoft Press.

Checklist voor de kwaliteitscontrole van een requirement

De checklist in tabel 4.3 kun je gebruiken voor het controleren van de kwaliteiten van een individuele requirement. Let op! Het is niet altijd noodzakelijk dat elke requirement aan al deze criteria (even zwaar) voldoet. De relevantie van een criterium is afhankelijk van de context, het doel en de doelgroep.

Tabel 4.3 Checklist voor een individuele requirement

Criterium		In orde	Verbeteren	Overbodig	Toelichting
De requirement:					
1.	Heeft slechts één eigenaar.	☐	☐	☐	
2.	Is relevant.	☐	☐	☐	
3.	Is helder beschreven.	☐	☐	☐	
4.	Is slechts op één manier op te vatten.	☐	☐	☐	
5.	Beschrijft slechts één behoefte, eis of restrictie.	☐	☐	☐	
6.	Is volledig.	☐	☐	☐	
7.	Is juist (<i>is vrij van logische fouten</i>).	☐	☐	☐	
8.	Bezit benodigde nauwkeurigheid.	☐	☐	☐	
9.	Is consistent (<i>sprekt zichzelf en andere requirements niet tegen</i>).	☐	☐	☐	
10.	Is verifieerbaar (<i>het is mogelijk om op basis van de requirement het product te meten, te toetsen of te testen</i>).	☐	☐	☐	
11.	Is realiseerbaar in de beoogde context.	☐	☐	☐	
12.	Heeft een prioriteit.	☐	☐	☐	
13.	Heeft een ID (identificer).	☐	☐	☐	
14.	Heeft een actuele status.	☐	☐	☐	
15.	Heeft een actueel versienummer.	☐	☐	☐	
16.	Is traceerbaar (<i>de bronnen evenals voorliggende documenten zijn vindbaar voor de belanghebbenden</i>).	☐	☐	☐	

Oefening

Je hebt nu kennis over requirements en de manier waarop je requirements schrijft. Deze oefening geeft je de mogelijkheid om je kennis toe te passen.

Een Requirements Management Tool selecteren voor je projecten

In eenvoudige projecten kun je requirements in een spreadsheet of een tekstverwerker vastleggen en bijhouden. Spreadsheets en tekstverwerkers bieden helaas niet voldoende ondersteuning bij complexere projecten. Het wordt erg ingewikkeld om bijvoorbeeld de relaties tussen de requirements weer te geven of de requirements traceerbaar te maken.

Daarom is het raadzaam om bij complexere projecten 'Requirements Management Tools' te gebruiken. Deze tools ondersteunen het vastleggen en beheren van requirements. Ze maken de samenwerking in en tussen teams eenvoudiger. Er zijn diverse tools beschikbaar op de markt. In deze opdracht onderzoek je eerst wat Requirements Management Tools zijn. Vervolgens schrijf je requirements voor het selecteren van een Requirements Management Tool die past bij jouw werkwijze in ontwikkelprojecten. Daarna vraag je aan een medestudent om een tool te selecteren die voldoet aan jouw requirements.

Aanbevolen taken

- Taak 1 Onderzoek welke Requirements Management Tools tot de top 3 behoren als het gaat om tools voor gebruik in softwareontwikkelpromjecten.
- Taak 2 Verken de functies en de kwaliteitskenmerken van deze tools, zodat je weet wat je van een Requirements Management Tool kunt verwachten.
- Taak 3 Schrijf user stories vanuit jouw perspectief om een Requirements Management Tool te kunnen selecteren.
- Taak 4 Schrijf software requirements voor de bruikbaarheid van een Requirements Management Tool vanuit jouw perspectief om een tool te kunnen selecteren. Bruikbaarheid is uitgelegd in hoofdstuk 3 'ISO 25010 Kwaliteit van softwareproduct'.
- Taak 5 Een Requirements Management Tool moet veilig zijn om misbruik zoals bedrijfsspionage te voorkomen. Schrijf daarom ook software requirements voor de beveiligbaarheid van de Requirements Management Tool. Beveiligbaarheid is uitgelegd in hoofdstuk 3 'ISO 25010 Kwaliteit van softwareproduct'.
- Taak 6 Vraag aan een medestudent om de meest geschikte Requirements Management Tool voor je te selecteren op basis van de requirements die je bij taak 3 t/m 5 hebt opgesteld. Op deze manier krijg je feedback als je requirements verbeterd moeten worden.

5 Onethisch en manipulatief gedrag herkennen

Software is geld en macht. Hoe groter de rol van geld en macht hoe groter het risico dat mensen onethisch en manipulatief gedrag vertonen. Je kunt software inzetten om mensen te helpen en eerlijk geld te verdienen. Tegelijkertijd kun je software inzetten om het grote publiek te manipuleren en te gebruiken om vooral zelf daar beter van te worden. Hackers zijn niet de enige mensen die onethisch gedrag vertonen. In je werkproces kun je ook bewust of onbewust onethisch gedrag vertonen.

Voorbeeld

Je bent een senior consultant en werkt bij een dienstverlener op het gebied van consulting en ICT. Je hebt de opdracht om een overheidsinstelling te ondersteunen, zodat deze instelling een informatiesysteem en leverancier kan selecteren. Het selectieproces moet volgens een openbare aanbesteding. De overheidsinstelling moet de requirements vaststellen waaraan het systeem moet voldoen, zodat de leveranciers een bod kunnen doen. Het hele selectieproces moet transparant en eerlijk verlopen, omdat het project met gemeenschapsgeld wordt gefinancierd.

In het selectieproces moet je als een onafhankelijke informatieanalist en adviseur optreden. Dit houdt in dat bij jou de belangen van de overheidsinstelling en de maatschappij centraal moeten staan.

Vriendjespolitiek

De overheidsinstelling heeft zelf niet de experts in dienst die requirements kunnen vaststellen voor het systeem. Ze moeten deze experts inhuren. Hiervoor moet jij profielen omschrijven waaraan de experts moeten voldoen. Deze profielen worden opgenomen in vacatures. Je wilt ervoor zorgen dat de overheidsinstelling mensen uit jouw netwerk inhuurt, zodat je investeert in relaties die voor jouw persoonlijke en zakelijke doelen belangrijk zijn. Daarom ga je de inhoud van de cv's van jouw voorkeursrelaties gebruiken om de vacatures op te stellen. Op deze manier vergroot je de kans dat alleen jouw voorkeursrelaties voldoen aan de eisen voor de vacatures. Hierdoor bestaat de kans dat ongeschikte mensen invloed krijgen in de besluitvorming bij een overheidsinstelling. Dit gedrag is onethisch.

Voorkeursleverancier helpen bij openbare aanbesteding

Het is je gelukt om je opdrachtgever te manipuleren en een team samen te stellen dat bestaat uit je voorkeursrelaties. Nu wil je dat de overheidsinstelling jouw voorkeursleverancier kiest. Deze leverancier is voor jou belangrijk omdat je als spreker kunt optreden bij hun conferenties. Hierdoor krijg je meer bekendheid en invloed bij de opdrachtgevers en andere leveranciers. Soortgelijke conferenties worden vaak bij luxe oorden gehouden en gefinancierd door leveranciers.

Bij het verzamelen en vaststellen van requirements ga je de opdrachtgever, gebruikers en andere belanghebbenden manipuleren, zodat ze de functies en kenmerken van het systeem van jouw voorkeursleverancier als requirements omschrijven. Dit doe je door je vragen over de requirements op zo'n manier te formuleren, zodat de antwoorden het systeem van je voorkeursleverancier omschrijven. Je zorgt er ook voor dat ze de

tekortkomingen van de systemen van de concurrenten van je voorkeursleverancier als verplichte eisen opgeven. Hierdoor voorkom je dat de overheidsinstelling de systemen van de concurrenten van je voorkeursleverancier selecteert. Het selectieproces verloopt oneerlijk en niemand kan bewijzen dat je het proces hebt gemanipuleerd. Je kiest eerst voor de belangen van jezelf en van je netwerk. De belangen van de overheidsinstelling die jou met publiek geld betaalt, komen in de tweede plaats. Dit gedrag is onethisch.

Oefeningen

Oefening 1

Deze oefening is bedoeld om jou als ICT'er bewust te maken van de ethische dilemma's die je in de praktijk kunt tegenkomen.

Aanbevolen taken

- Taak 1 Zoek op het internet drie artikelen of documentaires over onethisch gedrag bij bedrijven en instellingen waarbij ICT'ers actief hebben meegedaan of hebben gezwegen over de misstanden. Denk bijvoorbeeld aan de sjoemelsoftware van Volkswagen. Deze software is niet per ongeluk ontstaan. ICT'ers hebben op basis van requirements software ontworpen, ontwikkeld, getest en ingebouwd. Figuur 5.1 toont een fragment van een artikel hierover in het blad AG Connect. Volkswagen is geen uitzondering.
- Taak 2 Beschrijf welke handelingen volgens jou onethisch waren, zoals deze, in de door jou gekozen artikelen of in de documentaires zijn weergegeven?
- Taak 3 Kies één van de artikelen als een casus en beschrijf hoe jij had gehandeld in de beschreven situatie? Had je meegedaan? Had je gezwegen over het onethisch gedrag om je (loop)baan niet in gevaar te brengen? Had je maatregelen genomen om onethisch gedrag te stoppen?
- Taak 4 Bestudeer op de volgende website 'The Software Engineering Code of Ethics and Professional Practice':
 - > <https://www.computer.org/education/code-of-ethics>
- Taak 5 Maak een lijst van ethische regels die je in je projecten wilt toepassen.



Figuur 5.1 Screenshot van artikel over sjoemelsoftware. Bron: <https://www.agconnect.nl>

Oefening 2

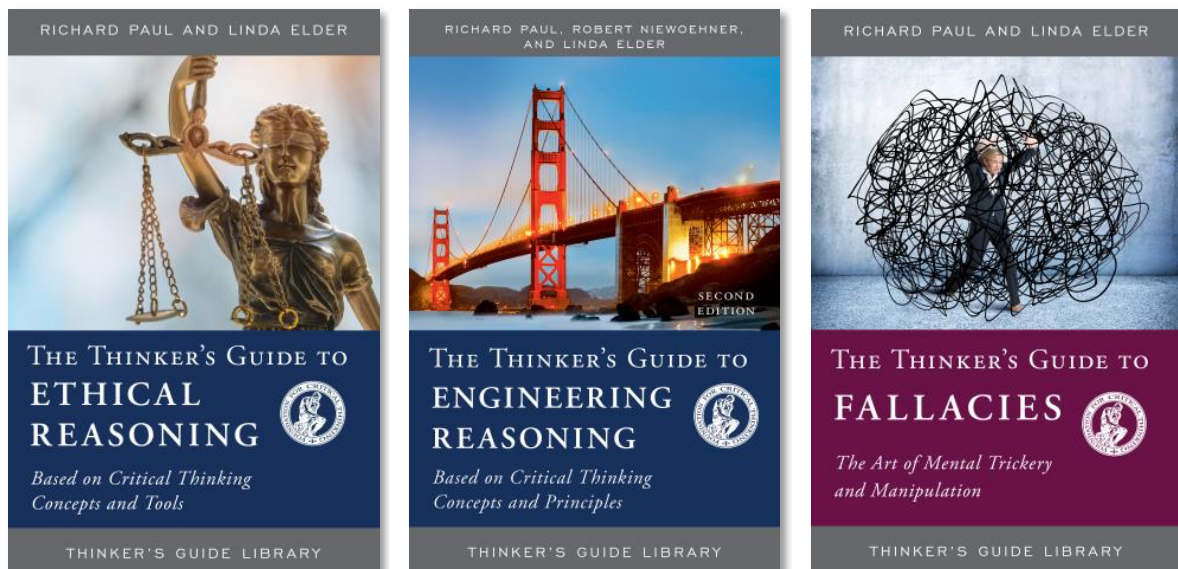
Je bent een ICT-student en je doet je afstudeerproject bij een zorgaanbieder. Je opdracht is om de zorgaanbieder te ondersteunen bij het selecteren van een systeem waarmee ze het personeel kunnen plannen voor de zorgaanvragen van de cliënten. Om het juiste systeem te kunnen selecteren wil je eerst de requirements van de belanghebbenden vaststellen. Vervolgens wil je een longlist opstellen van de potentiële systeemleveranciers. Bij het zoeken van potentiële systeemleveranciers voor de longlist kom je een leverancier tegen die recent jouw vader heeft afgewezen na een sollicitatiegesprek. Daarom beslis je om deze leverancier niet op te nemen op de longlist zodat de belanghebbenden deze niet kunnen selecteren.

1. Is het ethisch om de leverancier uit te sluiten die jouw vader heeft afgewezen na een sollicitatiegesprek?
2. Formulier twee argumenten voor je antwoord.

Additioneel studiemateriaal en naslagwerk

Indien nodig kun je de volgende bronnen gebruiken als studiemateriaal en naslagwerk:

- Paul, R. & Elder, L. (2019). *The Thinker's Guide to Ethical Reasoning: Based on Critical Thinking Concepts & Tools*. Lanham, MD: Rowman & Littlefield Publishers
- Paul, R., Niewoehner, R. & Elder, L. (2019). *The Thinker's Guide to Engineering Reasoning: Based on Critical Thinking Concepts & Tools*. Lanham, MD: Rowman & Littlefield Publishers
- Paul, R. & Elder, L. (2019). *The Thinker's Guide to Fallacies: The Art of Mental Trickery and Manipulation*. Lanham, MD: Rowman & Littlefield Publishers



Figuur 5.2 Aanbevolen bronnen voor zelfstudie

‘The Foundation for Critical Thinking’ bevat meer informatie en studiemateriaal over kritisch denken:

- > <http://www.criticalthinking.org>

'NSPE, National Society Of Professional Engineers' biedt bruikbare handreikingen over ethiek voor ingenieurs. Zie Code of Ethics for Engineers:

> <https://www.nspe.org/resources/ethics/code-ethics>

6 Vrije opdrachten gericht op HBO-i beroepstaak software analyseren

Dit hoofdstuk bevat drie vrije opdrachten. Deze opdrachten geven je de mogelijkheid om de verworven kennis, vaardigheden en inzichten toe te passen in projecten. De opdrachten zijn gerelateerd aan de HBO-i-stichting beroepstaak software analyseren op beheersingsniveau 1, 2 en 3. Deze hebben we overgenomen van de 'HBO-i-domeinbeschrijving 2018'. Figuur 6.1 toont de omslag van de domeinomschrijving. De volledige uitgave vind je op:

> <https://www.hbo-i.nl>

De HBO-i-stichting is de koepelorganisatie van hbo ICT-opleidingen in Nederland. De stichting zet zich in voor kennisuitwisseling en informatieoverdracht op vakinhoudelijk en onderwijskundig gebied. Volg je een ICT-opleiding op hbo-niveau? Dan is je opleiding hoogstwaarschijnlijk lid van de HBO-i. Op de website van de HBO-i vind je een overzicht van de aangesloten hogescholen.



Figuur 6.1 Omslag HBO-i-domeinbeschrijving 2018

Bruikbare informatiebronnen voor de opdrachten

De website van Reaco Academy bevat diverse vakinhoudelijke artikelen die relevant kunnen zijn voor de opdrachten in dit hoofdstuk en voor de praktijk:

> <https://www.reaco.nl>

Figuur 6.2 toont een screenshot waarin de inleidende teksten van twee artikelen als voorbeeld zijn weergegeven. De experts van Reaco Academy plaatsen regelmatig nieuwe artikelen over de actuele ontwikkelingen in het vakgebied requirements engineering.

	
Product owner vs business analist	Requirements in scrum van groot belang
Ben jij business analist en vervul je momenteel de rol van product owner? Of misschien heeft de product owner veel taken aan jou gedelegeerd, omdat hij er zelf geen tijd voor heeft. Hoe goed gaat je dat af? Als we de vergelijking product owner vs business analist...	In traditionele ICT-projecten stellen business / informatie analisten de requirements op tijdens een requirementsfase. Scrum kent geen requirementsfase en onderkent ook geen analistenrol. Dit betekent echter niet dat requirements in scrum onbelangrijk zijn....
LEES MEER	LEES MEER

Figuur 6.2 Artikelen over requirements engineering

De volgende boeken kunnen als naslagwerk dienen:

1. De Swart, N. (2017). *Handboek Requirements: Leidraad voor analisten in agile, traditionele en hybride omgevingen*. Tweede druk. Delft: Uitgeverij Eburon.
2. Laan, G.J. (2007). *Softwareontwikkeling met UML en Java*. Amsterdam: Pearson Education Benelux.
 - Dit boek is zeer vakkundig en helder geschreven. Het helpt je om de requirements te modeleren en de relatie met de code te zien.

Opdracht 1: User stories en acceptatiecriteria schrijven

Deze opdracht is gerelateerd aan de HBO-i beroepstaak software analyseren op beheersingsniveau 1. De beschrijving hiervan vind je in tekstvak 6.1.

Tekstvak 6.1 HBO-i beroepstaak software analyseren op beheersingsniveau 1

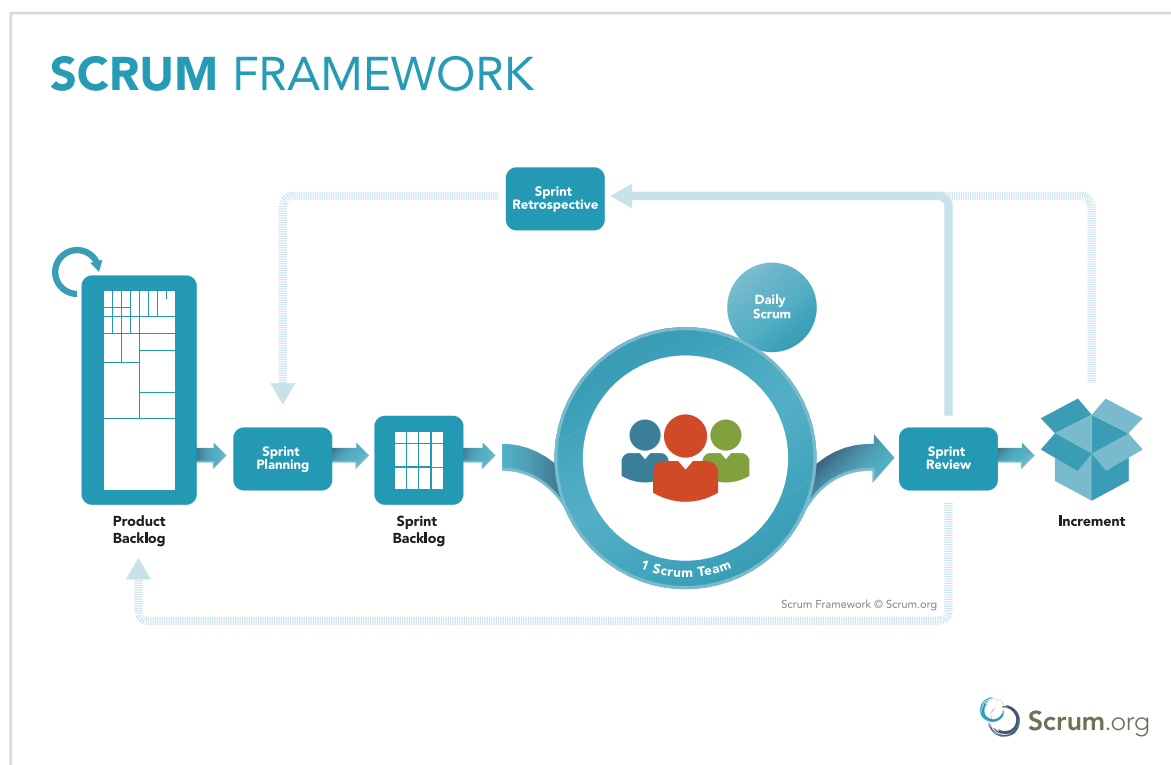
Beheersingsniveau 1

- Verzamelen en valideren van functionele eisen voor een softwaresysteem met één belanghebbende volgens een standaardmethode.
- Definiëren van acceptatiecriteria voor bovengenoemde functionele eisen.

In hoofdstuk 1 heb je kennis gemaakt met user stories en acceptatiecriteria. In agile ontwikkelmethoden zoals Scrum kun je user stories en acceptatiecriteria gebruiken om een product backlog samen te stellen. Figuur 6.3 toont de positie van product backlog in Scrum Framework. Opdracht 1 kun je toepassen in een project in het eerste jaar van je studie.

Aanbevolen taken

- Taak 1 Schrijf user stories voor een softwareproduct dat je in een project gaat ontwikkelen. Denk bijvoorbeeld aan een project in het eerste jaar van je studie.
- Taak 2 Schrijf voor elke user story acceptatiecriteria. Elke user story en elk acceptatiecriterium moet voor traceerbaarheid een unieke ID (identifier) krijgen. Zie US-ID en AC-ID in het volgende voorbeeld.



Figuur 6.3 Scrum Framework. Afbeelding gebruikt met toestemming van Scrum.org.

Indien nodig kun je meer informatie vinden over Scrum in 'The Scrum Guide' op de website hieronder. Er is ook een Nederlandse vertaling van beschikbaar.

> <https://www.scrum.org>

Voorbeeld

Topic	US-ID	User story	AC-ID	Acceptatiecriterium	Prioriteit
Betaalpas	US1	Als rekeninghouder wil ik de limiet voor betalen met betaalpas tijdelijk kunnen veranderen, zodat ik bij een pinautomaat aankoopbedragen kan betalen die hoger zijn dan mijn permanente betaallimiet.	US1-AC1	De bank-app heeft een optie voor 'limiet bepalen'.	Vereist
			US1-AC2	Op het scherm 'limiet bepalen' kan de rekeninghouder de limiet wijzigen.	Vereist
			US1-AC3	Op het scherm 'limiet bepalen' kan de rekeninghouder kiezen voor een tijdelijke of permanente limietwijziging.	Vereist
			US1-AC4	Op het scherm 'limiet bepalen' kan de rekeninghouder kiezen voor een start- en einddatum voor de tijdelijke limietwijziging.	Vereist

Use case beschrijving

Een user story geeft aan wat een gebruiker wil doen met het systeem en waarom. Je kunt de user story verrijken door hiervoor een use case beschrijving op te stellen (use case specification). Er zijn geen harde regels voor het opstellen van use case beschrijvingen. In bijlage C geven we een voorbeeld. Het voorbeeld is overgenomen van het volgende boek:

- Chamani, H., Kruijff, G., Oosting, G. & Van Rooyen, J. (2022). *Aan de slag met software testen: Principes, processen en technieken*. Derde druk. Amsterdam: Boom uitgevers.



Opdracht 2: Software requirements en acceptatiecriteria schrijven

Deze opdracht is gerelateerd aan de HBO-i beroepstaak software analyseren op beheersingsniveau 2. De beschrijving hiervan vind je in tekstvak 6.2.

Tekstvak 6.2 HBO-i beroepstaak software analyseren op beheersingsniveau 2

Beheersingsniveau 2	
	• Uitvoeren van een requirementsanalyse voor een softwaresysteem met verschillende belanghebbenden, rekening houdend met de kwaliteitseigenschappen waaronder security.
	• Uitvoeren van een analyse om functionaliteit, veiligheid, ontwerp, interfaces e.d. van een bestand systeem of bestaande component te formuleren en te valideren.
	• Opstellen van een acceptatietest aan de hand van kwaliteitseigenschappen.

In hoofdstuk 1 heb je kennis gemaakt met requirements in het algemeen. Je hebt geleerd wat acceptatiecriteria zijn. In hoofdstuk 2 heb je verder kennis verworven over software requirements. Je hebt geleerd wat functionele en niet-functionele software requirements zijn. In hoofdstuk 3 heb je je verdiept in ISO 25010 kwaliteit van softwareproduct voor het formuleren van niet-functionele software requirements.

Opdracht 2 is gericht op het schrijven van functionele en niet-functionele software requirements. Je leert ook acceptatiecriteria te formuleren voor deze requirements. De opdracht kun je toepassen in een project in het tweede jaar van je studie als voorbereiding op je stage. In het project ontwikkel je software voor een opdrachtgever buiten de opleiding. Je hebt te maken met meerdere belanghebbenden zoals diverse gebruikersgroepen en beheerders. Je docenten fungeren als begeleiders en beoordelaars.

Aanbevolen taken

- Taak 1 Schrijf functionele software requirements voor een product dat je gaat ontwikkelen voor een opdrachtgever buiten je opleiding.
- Taak 2 Schrijf acceptatiecriteria voor de functionele software requirements.
- Taak 3 Schrijf niet-functionele software requirements voor het softwareproduct. Gebruik hiervoor ISO 25010 kwaliteit van softwareproduct als uitgangspunt.
- Taak 4 Schrijf acceptatiecriteria voor de niet-functionele software requirements. Gebruik hiervoor ISO 25010 kwaliteit van softwareproduct als uitgangspunt.
- Taak 5 Verwerk de resultaten van taak 1 t/m 4 in een spreadsheet of in een Requirements Management Tool, zodat je de requirements beter kunt beheren.

Merk op dat een softwareproduct niet altijd aan alle kwaliteitskenmerken van ISO 25010 hoeft te voldoen. Een wegwerp-softwareproduct hoef je bijvoorbeeld niet langdurig te onderhouden. Daarom schrijf je geen requirements voor onderhoudbaarheid. Voor een programma zonder user interface schrijf je geen requirements voor bruikbaarheid. Bepaal samen met de belanghebbenden welke kwaliteitskenmerken relevant zijn.

Opdracht 3: Requirements opstellen voor je afstudeerproject

Deze opdracht is gerelateerd aan de HBO-i beroepstaak software analyseren op beheersingsniveau 3. De beschrijving hiervan vind je in tekstvak 6.3.

Tekstvak 6.3 HBO-i beroepstaak software analyseren op beheersingsniveau 3

Beheersingsniveau 3	
	• Uitvoeren van een requirementsanalyse voor een softwaresysteem met verschillende belanghebbenden in een context van bestaande systemen.
	• Definiëren van acceptatiecriteria aan de hand van kwaliteitseigenschappen en een uitgevoerde risicoanalyse met onder andere aandacht voor security aspecten.

Wil je dat je afstudeerdocenten jouw werk gericht kunnen beoordelen? Schrijf dan direct in de beginfase van het afstudeerproject requirements voor het beoogde softwareproduct of systeem dat je gaat ontwikkelen. Dit kun je doen door een SRS (Software Requirements Specification) te schrijven.

Een SRS is een document waarin een te ontwikkelen of een te selecteren softwareproduct is beschreven.

Een fictief afstudeerproject als voorbeeld

Je bent een ICT-student en je wilt beginnen met je afstudeerproject. Binnen je project ga je een nieuw softwareproduct ontwikkelen en toevoegen aan een bestaand systeem. Je onderzoekt onder andere de huidige situatie in het systeem, de gewenste situatie, het benodigde softwareproduct voor de nieuwe situatie en de requirements voor het softwareproduct. Deze requirements schrijf je in een SRS.

De requirements gebruik je als uitgangspunt voor het ontwikkelen van het softwareproduct en voor het waarborgen van de kwaliteit in het hele ontwikkelproject. De definitieve versie van de SRS voeg je toe aan je afstudeerdocumentatie. Tijdens de verdediging van je afstudeerproject ga je de meest relevante requirements in de SRS gebruiken om aan te tonen dat je het juiste softwareproduct op de juiste manier hebt ontwikkeld voor de belanghebbenden. De docenten en de andere betrokkenen bij je afstudeerproject kunnen de requirements als extra hulpmiddel gebruiken om je werk gericht te waarderen.

Aanbevolen taken

- Taak 1 Stel op basis van een vooronderzoek een inhoudsopgave op voor een SRS-document dat past bij jouw afstudeerproject. In 'Bijlage B: SRS-template voor een (afstudeer)project' vind je een voorbeeld. Je kunt de volgende zoektermen gebruiken als je op het Internet op zoek gaat naar een SRS-template: SRS template ISO/IEC/IEEE 29148:2011.
- Taak 2 Onderzoek de requirements voor het softwareproduct en schrijf de onderzochte requirements in het SRS-document. Het onderzoeks- en schrijfproces verloopt iteratief. Je hebt meerdere iteraties nodig om een goed SRS-document te kunnen schrijven.

- Taak 3 Laat je opdrachtgever en afstudeerbegeleider bij het bedrijf je SRS-document regelmatig reviewen, zodat jullie een expliciet en op elkaar afgestemd uitgangspunt creëren.
- Taak 4 Laat je afstudeerdocent(en) je SRS-document reviewen en aangeven welke requirements ze belangrijk vinden voor je beoordeling.
- Taak 5 Bereid een demonstratie voor op basis van de belangrijkste requirements, zodat je tijdens je afstudeerverdediging kunt aantonen dat je het juiste product op de juiste manier hebt ontwikkeld voor de belanghebbenden (validatie en verificatie).

Aandachtspunt

CleanText Academy heeft voor opdracht 3 de volgende workshop ontwikkeld: *Software requirements specificeren: De workshop voor afstudeerders*. Deze workshop kun je gratis downloaden op CleanText Academy:

> <https://cleantext-academy.blogspot.com>



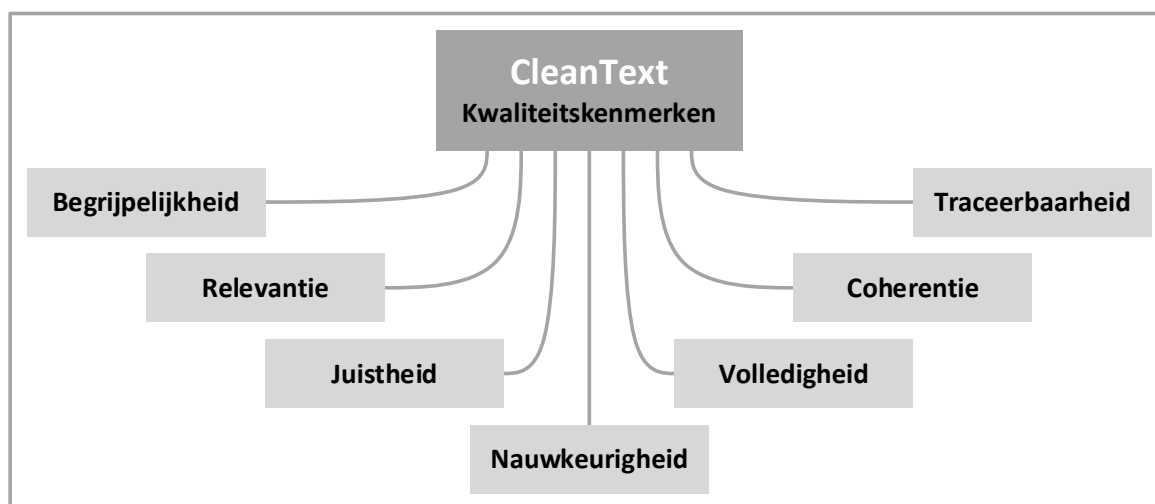
Bijlage A: CleanText kwaliteitskenmerken

Als ICT'er schrijf je technische teksten omdat je kennis wilt overdragen aan belanghebbenden van informatiesystemen. Het schrijven van deze teksten is geïntegreerd in taken zoals:

- Requirements analyseren
- Ontwerp maken
- Code schrijven
- Software testen
- Incidenten afhandelen
- Software beheren.

Fouten, onduidelijkheden of andere problemen in technische teksten kunnen leiden tot foute handelingen in het ontwikkel- of gebruiksproces van software. Dit kan fatale gevolgen hebben voor gebruikers, opdrachtgevers, leveranciers en andere betrokkenen. Daarom moet elke technische tekst aan specifieke kwaliteitskenmerken voldoen. Deze kenmerken bepaal je vanuit het perspectief van de lezers en andere belanghebbenden.

In dit boek definiëren we zeven kwaliteitskenmerken voor gegevens en informatie als uitgangspunt voor het creëren van technische teksten. Deze kenmerken zijn weergegeven in figuur BA.1. We hebben het aantal bewust beperkt tot zeven. Hierdoor willen we focussen op de meest fundamentele kwaliteitskenmerken voor het creëren van bruikbare technische teksten in de levenscyclus van software.



Figuur BA.1 *CleanText kwaliteitskenmerken*

Tabel BA.1 geeft een overzicht van CleanText kwaliteitskenmerken en de beschrijvingen ervan. Deze kenmerken kun je gebruiken voor het voorbereiden en uitvoeren van schrijftaken evenals voor het controleren en verbeteren van de resultaten.

Als ICT'er hoef je niet in elke situatie alle zeven kwaliteitskenmerken toe te passen. Je gebruikt wat jij daadwerkelijk nodig hebt voor jouw schrijftaak en je voegt extra

kwaliteitskenmerken toe wanneer je het nodig vindt. Deze kenmerken hoef je niet overal in een document even zwaar toe te passen. Je maakt keuzes op basis van de informatiebehoefte van de beoogde gebruikers en de context. Een onervaren gebruiker heeft bijvoorbeeld meer informatie nodig dan een expert.

Voorbeeld

Je bent een tester en in deze rol wil je requirements schrijven voor een prototype dat je als wegwerpsoftware wilt gebruiken voor een eenmalige test. Deze requirements schrijf je voor jezelf en voor een ontwikkelaar om samen het prototype te ontwikkelen en te testen.

Jullie werken in hetzelfde gebouw. In dit geval stellen jullie geen zware eisen aan de kwaliteit van de geschreven requirements. Deze requirements zijn slechts bedoeld als basisinformatie om een gezamenlijk vertrekpunt te hebben. Details communiceren jullie mondeling.

Tabel BA.1 De zeven CleanText kwaliteitskenmerken en hun beschrijvingen

#	Kwaliteitskenmerk	Beschrijving
1	Begrijpelijkheid	De beschikbare gegevens en informatie zijn helder (<i>de beoogde gebruikers kunnen deze juist interpreteren</i>).
2	Relevantie	De beschikbare gegevens en informatie zijn nuttig en zijn vrij van overbodige elementen.
3	Juistheid	De beschikbare gegevens, informatie en redeneringen zijn vrij van fouten.
4	Nauwkeurigheid	De beschikbare gegevens en informatie zijn accuraat.
5	Volledigheid	Alle noodzakelijke gegevens en informatie zijn beschikbaar voor de beoogde gebruikers.
6	Coherentie	De gegevens en informatie hebben een ordelijke samenhang, passend bij de denk- en werkwijze van de beoogde gebruikers.
7	Traceerbaarheid	De bronnen van gegevens en informatie evenals voorliggende gegevens en informatie zijn vindbaar voor de belanghebbenden.

Bron:

Chamani, H. (2021). *CleanText 2.0: De technische schrijfgids voor ICT'ers, coaches en trainers*. CleanText Academy.

Bijlage B: SRS-template voor een (afstudeer)project

Een SRS (Software Requirements Specification) is een document waarin de requirements voor een te ontwikkelen of een te selecteren softwareproduct of systeem zijn omschreven. In je afstudeerproject onderzoek je de requirements voorafgaand aan of in de startfase van je project als software het eindproduct is van het project. De onderzochte requirements leg je vast in een SRS. De SRS dient dan als uitgangspunt voor het ontwerp, de code en de test van het softwareproduct. De SRS is ook één van de informatiebronnen die de afstudeercommissie kan gebruiken om jouw softwareproduct en technische communicatie te beoordelen.

De SRS is een levend document. Bij voortschrijdend inzicht kun je de SRS aanpassen, zodat deze goed genoeg is voor je afstudeerportfolio of als bijlage bij je scriptie.

Een template bevat een herbruikbare structuur voor een specifiek type document. Je maakt een template, zodat je efficiënter kunt documenteren. In deze bijlage bieden we een structuur voor een SRS. Deze structuur kun je niet altijd letterlijk overnemen. Je moet deze eerst aanpassen en geschikt maken voor je project.

Gebruikte bronnen voor bijlage B: SRS-template voor een (afstudeer)project

- Arendsen, M., Cannegieter, J.J., Grund, A., Heck, P., De Klerk, S., & Zandhuis, J. (2008). *Succes met de requirements!: Ontwikkeling, validatie en beheer van requirements voor informatiesystemen*. Den Haag: Sdu uitgevers.
- Laplante, P.A. (2018). *Requirements engineering for software and systems*. Third Edition. Boca Raton: CRC Press.

1 Inleiding

1.1 Doelgroep en doel

<Beschrijf hier wie de beoogde lezers zijn van de SRS en wat ze willen bereiken.>

1.2 Scope

<Noem en beschrijf in het kort voor welk softwareproduct of systeem je de SRS schrijft. Denk bijvoorbeeld aan het beschrijven van het type softwareproduct of systeem en de belangrijkste functionaliteit of functies. Maak eventueel gebruik van schema's om de informatie te visualiseren.>

1.3 Referenties

<Maak een lijst van de relevante documenten en informatiebronnen die je als uitgangspunt hebt gebruikt om de SRS op te stellen. Denk bijvoorbeeld aan de documentatie van het systeem in de huidige situatie, interviewverslagen, enquêtes en feedbackformulieren.

Maak ook een lijst van de voorliggende documenten. Denk bijvoorbeeld aan het ontwerp en testspecificaties.>

2 Overall omschrijving

2.1 Systeemcontext

< Beschrijf welke relatie het systeem met de doelstellingen van het afstudeerbedrijf heeft. Welke bijdrage levert het systeem aan deze doelstellingen?

Noem en beschrijf de gebruikersgroepen van het systeem, de primaire functies en de relatie met andere systemen. Je kunt een 'use case diagram' gebruiken om deze informatie te visualiseren. Beschrijf ook het wat, waarom en hoe van het use case diagram.

Heeft het systeem geen user interface en gebruikers die direct ermee communiceren? Gebruik dan een 'context diagram' als alternatief voor een use case diagram.>

2.2 Productieomgeving

< Beschrijf de operationele omgeving waarin het systeem in gebruik wordt genomen. Denk hierbij aan hardware, operating systemen, databases, servers, websites en geografische verspreidingen van de gebruikers.

Je kunt eventueel een 'deployment diagram' gebruiken om de productieomgeving te visualiseren. Beschrijf ook het wat, waarom en hoe van het deployment diagram.>

2.3 Restricties voor ontwerp en realisatie

<Noem en beschrijf hier de restricties voor het ontwerp en de realisatie van het systeem. Denk bijvoorbeeld aan de keuze voor het type database of programmeertaal.>

2.4 Aannames en afhankelijkheden

< Beschrijf de aannames die invloed kunnen hebben op de requirements in deze SRS. Je neemt bijvoorbeeld aan dat jouw systeem bedoeld is voor een eenmalig onderzoek naar de mogelijkheden en beperkingen van een nieuwe technologie. Het systeem is een prototype en zal niet in een productieomgeving werken. Daarom zal je in dit geval geen requirements opstellen voor onderhoudbaarheid.

Beschrijf de afhankelijkheden die invloed hebben op deze SRS. Denk bijvoorbeeld aan (deel)systemen die door andere teams of partijen moeten worden geleverd.>

3 Functionele software requirements

<Beschrijf de functionele software requirements. Merk op dat je deze requirements in meerdere paragrafen en sub-paragrafen kunt clusteren. Paragraaf 3.1 kan bijvoorbeeld de functionele software requirements voor proces A bevatten en paragraaf 3.2 voor proces B enzovoort.

Voorbeeld

3.1 Proces A

3.2 Proces B

3.n ...

Maak eventueel functionele decomposities om de functionele software requirements te clusteren en te visualiseren.>

4 Kwaliteitskenmerken (niet-functionele software requirements)

Noem en beschrijf de kwaliteitskenmerken waaraan het softwareproduct of systeem moet voldoen. Merk op dat je deze requirements in meerdere paragrafen en sub-paragrafen kunt clusteren zoals weergegeven bij ISO 25010.

Voorbeeld

4.1 Functionele geschiktheid

4.2 Prestatie-efficiëntie

4.3 Uitwisselbaarheid

4.n ...

5 Overige requirements

<Beschrijf hier de overige requirements. Denk hierbij aan data requirements of requirements voor user interface, software-interface en hardware-interface.>

Bijlage A: Verklarende woordenlijst

<Stel hier een verklarende woordenlijst op. Deze kunnen bijvoorbeeld bedrijfs- of domeinspecifieke woorden zijn.>

Bijlage B: Visuele modellen voor requirements

<In deze bijlage kun je de visuele modellen opnemen voor de requirements. Denk aan Class Diagrams, Activity Diagrams, Sequence Diagrams enzovoort.>

De volgende boeken kun je eventueel als naslagwerk gebruiken voor visuele modellen:

- Beatty, J. & Chen, A. (2012). *Visual models for software requirements*. Redmond, WA: Microsoft Press
- Miles, R., & Hamilton, K. (2006). *Learning UML 2.0: A Practical Introduction to UML*. Sebastopol, CA: O'Reilly Media, Inc.

Bijlage C: Voorbeeld use case beschrijving

Tabel BC.1 Use case beschrijving wijzig pincode

Naam use case	Wijzig pincode
Samenvatting	Een rekeninghouder wil de pincode van een (nieuwe) bankpas wijzigen
Actor	Rekeninghouder
Systeem	Geldautomaat
Precondities	Geldautomaat wacht op een bankpas
Hoofdscenario	1. Rekeninghouder voert de bankpas in, in de geldautomaat
	2. Geldautomaat controleert de bankpas en vraagt naar de pincode
	3. Rekeninghouder voert de juiste pincode in
	4. Geldautomaat controleert de pincode en presenteert een keuzemenu
	5. Rekeninghouder kiest: Pincode wijzigen
	6. Geldautomaat geeft aan: Voer de nieuwe pincode in
	7. Rekeninghouder voert de nieuwe pincode in
	8. Geldautomaat geeft aan: Voer de nieuwe pincode nog een keer in
	9. Rekeninghouder voert de nieuwe pincode nog een keer in
	10. Geldautomaat vergelijkt de ingevoerde pincodes, wijzigt de pincode en presenteert een keuzemenu
	11. Rekeninghouder kiest: Stop
	12. Geldautomaat gaat naar toestand: Wacht op een bankpas
Alternatieve scenario's	1.a.1 Rekeninghouder voert een ongeldige pas in (geen bankpas)
	1.a.2 Geldautomaat weigert de kaart, geeft de kaart terug en genereert de volgende foutmelding: Voer een geldige bankpas in
	1.b.1 Rekeninghouder voert een pas in die als gestolen is opgegeven
	1.b.2 Geldautomaat controleert de bankpas, herkent deze als gestolen en slikt de bankpas in
	n ...
Postcondities	Geldautomaat wacht op een bankpas

Bron:

Chamani, H., Kruijff, G., Oosting, G., & Van Rooyen, J. (2022). *Aan de slag met software testen: Principes, processen en technieken*. Derde druk. Amsterdam: Boom uitgevers.

Bibliografie

Beatty, J., & Chen, A. (2012). *Visual models for software requirements*. Redmond, WA: Microsoft Press.

Black, R., Van Veenendaal, E., & Graham, D. (2012). *Foundation of Software Testing ISTQB Certification*. Hampshire: Cengage Learning EMEA.

Bouman, E. (2008). *SmarTEST: Slim testen van informatiesystemen*. Tweede druk. Den Haag: Sdu uitgevers.

Cannegieter, J., De Swart, N., & Zandhuis, J. (2014). *Grip op Requirements: IREB foundation examenstof uitgelegd en praktisch gemaakt*. Delft: Uitgeverij Eburon.

Chamani, H. (2011). *Analytisch denken en helder schrijven: Een trainingsboek voor ICT'ers en studenten*. Den Haag: Sdu uitgevers.

Chamani, H. (2021). *CleanText 2.0: De technische schrijfgids voor ICT'ers, coaches en trainers*. CleanText Academy.

Chamani, H., Kruijff, G., Oosting, G., & Van Rooyen, J. (2022). *Aan de slag met software testen: Principes, processen en technieken*. Derde druk. Amsterdam: Boom uitgevers.

De Best, B. (2006). *Acceptatiecriteria*. Den Haag: Sdu uitgevers.

De Swart, N. (2017). *Handboek Requirements: Leidraad voor analisten in agile, traditionele en hybride omgevingen*. Tweede druk. Delft: Uitgeverij Eburon.

Laan, G.J. (2007). *Softwareontwikkeling met UML en Java*. Amsterdam: Pearson Education Benelux.

Laplante, P.A. (2018). *Requirements engineering for software and systems*. Third Edition. Boca Raton: CRC Press.

Miles, R., & Hamilton, K. (2006). *Learning UML 2.0: A Practical Introduction to UML*. Sebastopol, CA: O'Reilly Media, Inc.

Paul, R., & Elder, L. (2019). *The Thinker's Guide to Ethical Reasoning: Based on Critical Thinking Concepts & Tools*. Lanham, MD: Rowman & Littlefield Publishers.

Paul, R., & Elder, L. (2019). *The Thinker's Guide to Fallacies: The Art of Mental Trickery and Manipulation*. Lanham, MD: Rowman & Littlefield Publishers.

Paul, R., Niewoehner, R., & Elder, L. (2019). *The Thinker's Guide to Engineering Reasoning: Based on Critical Thinking Concepts & Tools*. Lanham, MD: Rowman & Littlefield Publishers.

Pichler, R. (2017). *Strategize: Product Strategy and Product Roadmap Practices for the Digital Age*. Pichler Consulting.

Pohl, K., & Rupp, C. (2016). *Requirements Engineering Fundamentals: A Study Guide for the Certified Professional for Requirements Engineering Exam - Foundation Level - IREB Compliant*. San Rafael, CA: Rocky Nook Inc.

Schotanus, C. (2008). *TestFrame: Een methode voor gestructureerd testen*. Den Haag: Sdu uitgevers.

Van Bart, P., & Steehouder, M. (2008). *Basisboek technische communicatie*. Assen: Van Gorcum.

Wiegers, K., & Beatty, J. (2013). *Software Requirements*. Third Edition. Redmond. WA: Microsoft Press.

Geraadpleegde websites

- > <https://www.agconnect.nl>
- > <https://www.computer.org/education/code-of-ethics>
- > <http://www.criticalthinking.org>
- > <https://www.gripoprequirements.nl>
- > <https://www.hbo-i.nl>
- > <https://www.ireb.org/en>
- > <https://iso25000.com>
- > <https://www.nspe.org/resources/ethics/code-ethics>
- > <https://www.reaco.nl>
- > <https://www.scrum.org>
- > <http://www.smartest.nl>

Over de auteur

Hossein Chamani heeft ervaring in functies en rollen als docent, trainer, projectmanager, testmanager, network integrator en network manager. Sinds 2001 werkt hij als vakdocent bij de opleiding Informatica van Hogeschool Rotterdam. Hij is de oprichter van CleanText Academy. Daarnaast is hij één van de vier auteurs van het boek 'Aan de slag met software testen'.

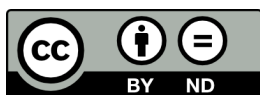


Download gratis boeken en workshops

CleanText Academy publiceert boeken en workshops op het gebied van technische communicatie voor ICT-studenten en ICT'ers in opleiding. De auteurs hebben geen financiële of commerciële belangen. Alle publicaties zijn gratis te downloaden op:

> <https://cleantext-academy.blogspot.com>





Dit boek valt onder de volgende Creative Commons-licentie:

- Naamsvermelding-GeenAfgeleideWerken 4.0 Internationaal (CC BY-ND 4.0).

Je bent vrij om het werk te delen — te kopiëren, te verspreiden en door te geven via elk medium of bestandsformaat voor alle doeleinden, inclusief commerciële doeleinden.

De licentiegever kan deze toestemming niet intrekken zolang aan de licentievoorwaarden voldaan wordt.

Onder de volgende voorwaarden:

- **Naamsvermelding** — De gebruiker dient de maker van het werk te vermelden, een link naar de licentie te plaatsen en aan te geven of het werk veranderd is. Je mag dat op redelijke wijze doen, maar niet zodanig dat de indruk gewekt wordt dat de licentiegever instemt met je werk of je gebruik van het werk.
- **GeenAfgeleideWerken** — Je mag het veranderde materiaal niet verspreiden als je het werk hebt geremixt, veranderd, of op het werk hebt voortgebouwd.
- **Geen aanvullende restricties** — Je mag geen juridische voorwaarden of technologische voorzieningen toepassen die anderen er juridisch in beperken om iets te doen wat de oorspronkelijke licentie wel toestaat.

De volledige licentie kun je lezen op de volgende webpagina:

- > <https://creativecommons.org/licenses/by-nd/4.0/legalcode.nl>

Omslag en opmaak binnenwerk: Hossein Chamani

Versie 1.0: 28 maart 2020, CleanText Academy

Versie 1.1: 10 maart 2021, CleanText Academy

Versie 1.2: 30 januari 2022, CleanText Academy

<https://cleantext-academy.blogspot.com>

Contact: cleantext.ict@gmail.com

CleanText Academy

Hossein Chamani